

Lucas Alexandre Beckhauser

COMPARAÇÃO DE DESEMPENHO ENTRE AS ARQUITETURAS DE COMUNICAÇÃO REST E GRPC

Trabalho de conclusão de curso apresentado
como requisito parcial para a obtenção do
grau de bacharel em Sistemas de Informação
do Instituto Federal Catarinense.

Instituto Federal Catarinense – IFC

Câmpus Araquari

Bacharelado em Sistemas de Informação

Orientador: Prof. Dr. Eduardo Da Silva

Araquari – SC

Agosto de 2024

Lucas Alexandre Beckhauser

COMPARAÇÃO DE DESEMPENHO ENTRE AS ARQUITETURAS DE COMUNICAÇÃO REST E GRPC/ Lucas Alexandre Beckhauser. – Araquari – SC, Agosto de 2024-

57 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof. Dr. Eduardo Da Silva

Monografia (Graduação) – Instituto Federal Catarinense – IFC

Câmpus Araquari

Bacharelado em Sistemas de Informação, Agosto de 2024.

I. Arquitetura de Software. II. REST. III. gRPC. IV. Comparação de Desempenho
V. Consumo de Recursos

Lucas Alexandre Beckhauser

COMPARAÇÃO DE DESEMPENHO ENTRE AS ARQUITETURAS DE COMUNICAÇÃO REST E GRPC

Trabalho de conclusão de curso apresentado
como requisito parcial para a obtenção do
grau de bacharel em Sistemas de Informação
do Instituto Federal Catarinense.

Araquari – SC, 24 de agosto de 2024:

Prof. Dr. Eduardo Da Silva
Orientador

Prof. Dr. Marco Antonio Torrez Rojas
Convidado 1

Prof. Me. Marco André Lopes Mendes
Convidado 2

Araquari – SC
Agosto de 2024

Dedico este trabalho a todos que me ajudaram ao longo desta jornada. Aos meus pais, por todas as vezes que abriram mão de algo em favor do meu futuro. À minha namorada e aos meus amigos, pelo apoio constante em todos os momentos.

Agradecimentos

Sou muito grato aos meus pais, por sempre apoiarem minhas decisões ao longo do caminho que me trouxe até aqui. Agradeço também ao IFC, que me ajudou a definir a direção do meu futuro profissional, e a todos os professores que sempre estiveram presentes para dar conselhos e esclarecer dúvidas. Também agradeço aos meus amigos, que tornaram essa jornada mais leve e divertida.

“Fix the cause, not the symptom.”
(Maguire, Steve)

Resumo

Este trabalho apresenta uma análise comparativa entre o desempenho das arquiteturas de comunicação REST e gRPC para auxiliar na tomada de decisão sobre qual arquitetura empregar em um projeto de software, foram desenvolvidas duas APIs em Python, uma utilizando REST e outra utilizando gRPC, ambas operando sobre um banco de dados simulado em JSON. Foram realizados testes de desempenho para definir um ponto de referência para comparação entre as duas arquiteturas em diferentes cenários. Os testes consideraram quatro cenários: obtenção de poucos registros, obtenção de muitos registros, inserção com *payload* pequeno e inserção com *payload* grande. Cada teste envolveu dois volumes de dados: 20 e 100 usuários virtuais realizando requisições às APIs, coletando dados sobre o funcionamento em três métricas: tempo de resposta das requisições, uso de CPU e consumo de memória. Os resultados mostraram que a arquitetura gRPC apresentou tempos de resposta significativamente menores e menor uso de CPU e memória em comparação com a arquitetura REST em todos os cenários testados. A gRPC demonstrou ser mais eficiente e consistente, especialmente em ambientes de alta demanda. Por outro lado, a arquitetura REST, embora mais simples e amplamente adotada, apresentou maiores tempos de resposta e maior consumo de recursos, o que pode ser um fator limitante em aplicações de alta demanda. Assim, a arquitetura gRPC é recomendada para os cenários específicos testados que exigem alto desempenho e eficiência no uso de recursos, enquanto a REST pode não ser a melhor escolha dentro dos parâmetros avaliados neste estudo.

Palavras-chave: Arquitetura de Software; REST; gRPC; Comparação de Desempenho; Consumo de Recursos;

Abstract

This study presents a comparative analysis of the performance of REST and gRPC communication architectures to assist in the decision-making process of which architecture to employ in a software project, two APIs were developed in Python, one using REST and the other using gRPC, both operating on a simulated JSON database. Performance tests were conducted to establish a reference point for comparison between the two architectures in different scenarios. The tests were conducted in four scenarios: fetching few records, fetching many records, inserting with small payload, and inserting with large payload. Each test involved two volumes of data: 20 and 100 virtual users making requests to the APIs, collecting data on three performance metrics: request response time, CPU usage, and memory consumption. The results showed that the gRPC architecture had significantly lower response times and lower CPU and memory usage compared to the REST architecture in all tested scenarios. gRPC proved to be more efficient and consistent, especially in high-demand environments. On the other hand, the REST architecture, though simpler and widely adopted, showed higher response times and greater resource consumption, which can be a limiting factor in high-demand applications. Therefore, the gRPC architecture is recommended for the specific tested scenarios requiring high performance and efficient resource usage, while REST may not be the best choice within the parameters evaluated in this study.

Keywords: Software Architecture; REST; gRPC; Performance Comparison; Resource Consumption;

Lista de ilustrações

Figura 1 – Definição do objeto Livro no <i>Protocol Buffers</i>	20
Figura 2 – Definição do <i>service</i> Biblioteca no <i>Protocol Buffers</i>	20
Figura 3 – Definição do <i>payload</i> livro em JSON.	22
Figura 4 – Exemplo de objeto JSON representando uma biblioteca	23
Figura 5 – Estrutura do banco de dados que será consumido por ambas as APIs	27
Figura 6 – Fluxo de Requisição e Resposta da API gRPC no JMeter	28
Figura 7 – Fluxo de Requisição e Resposta da API REST no JMeter	29
Figura 8 – Definição de Livro no <i>Protocol Buffers</i>	32
Figura 9 – Definição dos <i>services</i> no <i>Protocol Buffers</i>	32
Figura 10 – Definição do método GetAll dentro no <i>Protocol Buffers</i>	32
Figura 11 – Definição do método GetOne dentro no <i>Protocol Buffers</i>	33
Figura 12 – Definição do método PostOne no <i>Protocol Buffers</i>	33
Figura 13 – Definição do método PostOne no <i>Protocol Buffers</i>	33
Figura 14 – Exemplo de requisição para o <i>endpoint</i> POST /books	35
Figura 15 – Exemplo de requisição para o <i>service</i> PostOne	36
Figura 16 – Exemplo de requisição para o <i>endpoint</i> POST /books/list	36
Figura 17 – Exemplo de requisição para o <i>service</i> PostList	37
Figura 18 – Cenários para a Comparação das APIs REST e gRPC que serão Testados.	38
Figura 19 – Tempo de resposta para teste com obtenção de um registro em amostra de 20 usuários.	39
Figura 20 – Consumo de CPU para teste com obtenção de um registro em amostra de 20 usuários.	40
Figura 21 – Consumo de memória para teste com obtenção de um registro em amostra de 20 usuários.	40
Figura 22 – Tempo de resposta para teste com obtenção de um registro em amostra de 100 usuários.	41
Figura 23 – Consumo de CPU para teste com obtenção de um registro em amostra de 100 usuários.	41
Figura 24 – Consumo de memória para teste com obtenção de um registro em amostra de 100 usuários.	42
Figura 25 – Tempo de resposta para teste com obtenção de cinco mil registros em amostra de 20 usuários.	43
Figura 26 – Consumo de CPU para teste com obtenção de cinco mil registros em amostra de 20 usuários.	43
Figura 27 – Consumo de memória para teste com obtenção de cinco mil registros em amostra de 20 usuários.	44

Figura 28 – Tempo de resposta para teste com obtenção de cinco mil registros em amostra de 100 usuários.	44
Figura 29 – Consumo de CPU para teste com obtenção de cinco mil registros em amostra de 100 usuários.	45
Figura 30 – Consumo de memória para teste com obtenção de cinco mil registros em amostra de 100 usuários.	45
Figura 31 – Tempo de resposta para teste com inserção de um registro em amostra de 20 usuários.	46
Figura 32 – Consumo de CPU para teste com inserção de um registro em amostra de 20 usuários.	46
Figura 33 – Consumo de memória para teste com inserção de um registro em amostra de 20 usuários.	47
Figura 34 – Tempo de resposta para teste com inserção de um registro em amostra de 100 usuários.	47
Figura 35 – Consumo de CPU para teste com inserção de um registro em amostra de 100 usuários.	48
Figura 36 – Consumo de memória para teste com inserção de um registro em amostra de 100 usuários.	48
Figura 37 – Tempo de resposta para teste com inserção de uma lista de livros em amostra de 20 usuários.	49
Figura 38 – Consumo de CPU para teste com inserção de uma lista de livros em amostra de 20 usuários.	50
Figura 39 – Consumo de memória para teste com inserção de uma lista de livros em amostra de 20 usuários.	50
Figura 40 – Tempo de resposta para teste com inserção de uma lista de livros em amostra de 100 usuários.	51
Figura 41 – Consumo de CPU para teste com inserção de uma lista de livros em amostra de 100 usuários.	51
Figura 42 – Consumo de memória para teste com inserção de uma lista de livros em amostra de 100 usuários.	52
Figura 43 – Resultados dos testes de tempo de resposta para REST e gRPC	52
Figura 44 – Resultados dos testes de uso de CPU para REST e gRPC	53
Figura 45 – Resultados dos testes de uso de memória para REST e gRPC	53

Lista de abreviaturas e siglas

REST	<i>Representational State Transfer</i> - Transferência de Estado Representacional
gRPC	<i>Generic Remote Procedure Call</i> - Chamada de Procedimento Remoto Genérica
HTTP	<i>Hypertext Transfer Protocol</i> - Protocolo de Transferência de Hipertexto
CPU	<i>Central Processing Unit</i> - Unidade de Processamento Central
API	<i>Application Programming Interface</i> - Interface de Programação de Aplicação
JSON	<i>JavaScript Object Notation</i> - Notação de Objetos JavaScript
Protobuf	<i>Protocol Buffers</i> - Buffers de Protocolo
URL	<i>Uniform Resource Locator</i> - Localizador de Recursos Uniforme
ISBN	<i>International Standard Book Number</i> - Número de Livro Padrão Internacional
RAM	<i>Random Access Memory</i> - Memória de Acesso Aleatório
CRUD	<i>Create, Read, Update and Delete</i> - Criar, Ler, Atualizar e Excluir
Nº	Número

Sumário

1	INTRODUÇÃO	14
1.1	Motivação para a comparação entre REST e gRPC	14
1.2	Objetivos	15
1.3	Estrutura da Tese	15
2	FUNDAMENTAÇÃO TEÓRICA	17
2.1	Interface de Programação de Aplicação (API)	17
2.2	HTTP	17
2.3	Arquiteturas de Comunicação	19
2.3.1	gRPC	19
2.3.2	REST	21
2.4	Ferramentas para os Testes de Desempenho	23
2.4.1	JMeter	23
2.4.2	Docker	24
2.5	Considerações finais	24
3	METODOLOGIA PROPOSTA	26
3.1	Definição e Elaboração do Banco de Dados	26
3.1.1	Estrutura do Banco de Dados	26
3.2	Definição dos Testes no JMeter	27
3.2.1	Cenário de Teste gRPC	28
3.2.2	Cenário de Teste REST	28
3.3	Definição de Captura das Métricas	28
3.3.1	Comando Docker Utilizado	28
3.3.2	Parâmetros Monitorados	29
3.3.3	Procedimento de Captura de Métricas	29
3.4	API REST	30
3.4.1	Estrutura da API REST	30
3.4.1.1	Endpoint para buscar todos os livros disponíveis	30
3.4.1.2	Endpoint para buscar um livro específico pelo ID	30
3.4.1.3	Endpoint para adicionar um novo livro	30
3.4.1.4	Endpoint para adicionar uma lista de livros	31
3.5	API gRPC	31
3.5.1	Definição dos Serviços gRPC	31
3.5.1.1	Service para recuperar todos os livros (GetAll)	31
3.5.1.2	Service para recuperar um livro específico pelo ID (GetOne):	31

3.5.1.3	Service para adicionar um novo livro (PostOne)	32
3.5.1.4	Service para adicionar uma lista de novos livro (PostList)	33
3.6	Configuração dos Testes	34
3.6.1	Teste 1: Obtenção de Um Registro	34
3.6.1.1	Configuração do Teste	34
3.6.2	Teste 2: Obtenção de Cinco Mil Registros	34
3.6.2.1	Configuração do Teste	34
3.6.3	Teste 3: Inserção de Um Registro	35
3.6.3.1	Configuração do Teste:	35
3.6.4	Teste 4: Inserção de Dez Registros	35
3.6.4.1	Configuração do Teste:	36
3.7	Considerações Finais	37
4	RESULTADOS	38
4.1	Especificação	38
4.2	Teste 1: Obtenção de Um Registros	39
4.2.1	Amostra com 20 usuários	39
4.2.2	Amostra com 100 usuários	40
4.3	Teste 2: Obtenção de Cinco Mil Registros	42
4.3.1	Amostra com 20 usuários	42
4.3.2	Amostra com 100 usuários	42
4.4	Teste 3: Inserção de Um Registro	44
4.4.1	Amostra de 20 usuários	45
4.4.2	Amostra de 100 usuários	47
4.5	Teste 4: Inserção de Dez Registros	49
4.5.1	Amostra de 20 usuários	49
4.5.2	Amostra de 100 usuários	50
4.6	Considerações Finais	52
5	CONCLUSÃO	54
5.1	Trabalhos Futuros	54
	REFERÊNCIAS	56

1 Introdução

No contexto do desenvolvimento web, a escolha da arquitetura de comunicação desempenha um papel importante na determinação do desempenho geral e eficiência do sistema. Entre as diversas formas de se estabelecer essa comunicação, destacam-se o REST e o gRPC. O REST é amplamente utilizado para a comunicação entre serviços web devido à sua simplicidade e compatibilidade com HTTP. Já o gRPC é uma alternativa mais recente que utiliza o protocolo HTTP/2 e Protobuf para oferecer uma comunicação mais eficiente ([ŚLIWA; PAŃCZYK, 2021](#)).

A comparação de desempenho de arquiteturas de comunicação, particularmente entre gRPC e REST, tem recebido atenção significativa em pesquisas recentes. Mariuz?? apresentou uma comparação abrangente do desempenho de três arquiteturas de comunicação, incluindo REST, GraphQL e gRPC, usadas em aplicações web. Seu estudo fornece considerações significativas sobre a eficiência e eficácia dessas arquiteturas, trazendo à tona suas respectivas capacidades em aplicações do mundo real.

Além disso, Bolanowski[2022](#) exploraram os aspectos práticos da utilização de APIs REST e gRPC para tarefas de comunicação em ecossistemas baseados em microsserviços. Sua análise contribui para o entendimento das implicações operacionais dessas arquiteturas de comunicação em aplicativos modernos, adicionando relevância prática à comparação de desempenho entre gRPC e REST.

Berg[2023](#) realizaram um estudo sobre o *throughput* de requisições de gRPC em comparação com chamadas de API convencionais (REST), destacando que gRPC oferece melhor desempenho para cargas maiores de mensagens. De forma similar, Buzhin[2023](#) compararam o uso de gRPC e REST em sistemas de monitoramento de infraestrutura de redes virtualizadas, demonstrando que gRPC lida melhor com grandes volumes de dados e apresenta menor latência em várias situações de transferência.

Por este motivo, a comparação de desempenho entre gRPC e REST, juntamente com outras arquiteturas de comunicação, é uma área de pesquisa complexa que abrange vários tipos de estudos, análises práticas e considerações teóricas. Entender as características de desempenho e implicações operacionais dessas interfaces é essencial para a tomada de decisões informadas no desenvolvimento de aplicações web e no *design* de arquitetura.

1.1 Motivação para a comparação entre REST e gRPC

A escolha deste tema justifica-se pela crescente necessidade de sistemas de *software* eficientes e escaláveis em um cenário tecnológico que se encontra em constante evolução.

As arquiteturas de comunicação, como REST e gRPC, desempenham papéis fundamentais na determinação do desempenho, da escalabilidade e da eficiência dos sistemas distribuídos. Enquanto REST, uma arquitetura baseada em padrões web estabelecidos, é amplamente utilizada devido à sua simplicidade e compatibilidade com a infraestrutura da internet existente, gRPC, por outro lado, oferece uma abordagem moderna com foco na eficiência da comunicação e no desempenho por meio do uso de protocolo buffers e HTTP/2. Comparar o desempenho dessas duas arquiteturas não só esclarece suas vantagens e desvantagens práticas em diferentes cenários de aplicação, mas também orienta desenvolvedores e arquitetos de *software* na tomada de decisões informadas sobre qual arquitetura é mais adequada para atender às necessidades específicas de seus projetos. Dessa forma, investigar o desempenho entre REST e gRPC torna-se essencial para otimizar o desenvolvimento e a implementação de serviços web modernos, assegurando que as aplicações sejam não apenas funcionais, mas que tenham também um alto desempenho e escalabilidade.

1.2 Objetivos

Este trabalho tem como objetivo comparar as principais características relacionadas ao desempenho das arquiteturas REST e gRPC em um contexto de comunicação entre serviços. Para isso, foram desenvolvidas duas APIs de forma semelhante, garantindo que ambas consumissem dados de dois bancos de dados idênticos, e somente seguissem padrões distintos de arquitetura.

Para realizar a comparação, foram considerados três critérios principais: uso de CPU, consumo de memória e tempo de resposta. O objetivo é entender se, para esses critérios estabelecidos, as soluções oferecidas pela arquitetura gRPC são mais eficientes do que as soluções existentes na arquitetura REST.

Foram realizados testes de desempenho para responder a três perguntas fundamentais:

1. Qual arquitetura apresenta menor uso de CPU durante a execução de múltiplas requisições paralelas?
2. O gRPC consome menos memória em comparação ao REST ao realizar as mesmas operações de busca?
3. Qual arquitetura oferece menor tempo de resposta em operações de consulta intensiva?

1.3 Estrutura da Tese

Esta tese está organizada em cinco capítulos, além das referências:

- **Capítulo 1** - Introdução: Apresenta o contexto, a motivação e os objetivos do trabalho, com foco na comparação entre as arquiteturas REST e gRPC.
- **Capítulo 2** - Fundamentação Teórica: Explora os conceitos essenciais e as tecnologias envolvidas nas arquiteturas REST e gRPC.
- **Capítulo 3** - Metodologia Proposta: Descreve a abordagem adotada para a realização dos testes de desempenho e a coleta das métricas.
- **Capítulo 4** - Resultados: Apresenta e analisa os resultados obtidos nos testes comparativos das duas arquiteturas.
- **Capítulo 5** - Conclusão: Resume as principais conclusões e sugere direções para trabalhos futuros.

2 Fundamentação Teórica

Este tópico visa apresentar termos, conceitos e tecnologias essenciais para o entendimento deste trabalho. Serão explicados termos chave relacionados a APIs, detalhando como as arquiteturas REST e gRPC utilizam os protocolo HTTP para realizar solicitações, com os dados sendo transmitidos pela rede no formato JSON e proto. Além disso, serão discutidas as tecnologias que auxiliarão na realização dos testes de desempenho.

2.1 Interface de Programação de Aplicação (API)

Uma API (*Application Programming Interface*) é um conjunto de regras e especificações que os programas de *software* podem seguir para se comunicar entre si. Servindo como uma interface entre diferentes *softwares*, as APIs permitem que suas funcionalidades sejam utilizadas por outros aplicativos sem a necessidade de compartilhar o código interno.

As APIs são projetadas para abstrair a funcionalidade de um componente, oferecendo somente objetos ou ações que são necessários para o desempenho da tarefa e omitindo os demais detalhes de implementação. As APIs podem ser locais (quando as diferentes partes de um programa interagem para executar uma função) ou web APIs (que permitem a interação entre aplicativos e serviços na *Internet*). Elas são um componente indispensável no desenvolvimento moderno de aplicativos, permitindo um modelo de "plataforma como serviço" onde funcionalidades específicas são oferecidas por sistemas especializados, otimizando a interoperabilidade e a escalabilidade das aplicações (DOCS, 2024a).

Uma API define funcionalidades que são independentes de suas respectivas implementações, permitindo que suas funcionalidades sejam entendidas e utilizadas por diferentes aplicações. Elas permitem uma modularidade eficaz e a possibilidade de inovação sem um alto grau de dependência mútua entre sistemas. Por exemplo, uma API de geolocalização permite que você utilize o serviço de mapas de diferentes fornecedores sem ter que integrar cada um dos sistemas diretamente ao seu código (DOCS, 2024a).

2.2 HTTP

O *Hypertext Transfer Protocol* (HTTP) é um protocolo de aplicação utilizado para a transmissão de documentos de hipermídia, como HTML, na web. Ele foi projetado para a comunicação entre navegadores da web e servidores web, mas também pode ser utilizado para outros fins. O HTTP segue um modelo cliente-servidor, no qual um cliente abre uma

conexão para fazer uma solicitação e aguarda até receber uma resposta. Além disso, o HTTP é um protocolo sem estado, o que significa que o servidor não mantém nenhum dado entre duas solicitações (DOCS, 2024c). Os métodos HTTP definem a ação que se pretende realizar nos recursos especificados (DOCS, 2024b). Os métodos mais comuns são:

- *GET*: Solicita a representação de um recurso específico. Requisições usando *GET* devem apenas recuperar dados.
- *POST*: Utilizado para enviar uma entidade a um recurso específico, frequentemente causando uma mudança no estado do recurso ou efeitos colaterais no servidor.
- *PUT*: Substitui todas as representações atuais do recurso de destino pela carga de dados da requisição.
- *DELETE*: Remove um recurso específico.
- *PATCH*: Aplica modificações parciais a um recurso.

A evolução do HTTP ao longo dos anos levou ao desenvolvimento do HTTP/2, lançado em 2015, que introduz várias melhorias em relação ao seu antecessor, o HTTP/1.1, amplamente utilizado desde 1999 (BRITANNICA, 2024). As principais diferenças e melhorias entre o HTTP/1.1 e o HTTP/2 incluem:

- **Multiplexação**: O HTTP/2 permite que múltiplas requisições e respostas sejam enviadas simultaneamente pela mesma conexão TCP. No HTTP/1.1, cada requisição precisa ser enviada uma de cada vez ou através de múltiplas conexões.
- **Compressão de Cabeçalhos**: O HTTP/2 utiliza compressão de cabeçalhos para reduzir o tamanho dos cabeçalhos HTTP enviados. No HTTP/1.1, os cabeçalhos são enviados em texto puro.
- **Priorização**: O HTTP/2 permite que os clientes indiquem a prioridade das requisições, ajudando os servidores a determinar a ordem de processamento e resposta, melhorando a eficiência na entrega de conteúdo.
- **Server Push**: O HTTP/2 permite que os servidores enviem recursos adicionais ao cliente sem que o cliente os solicite diretamente, antecipando as necessidades do cliente e reduzindo o tempo de carregamento da página.
- **Manutenção da Conexão**: Em vez de abrir várias conexões, o HTTP/2 mantém uma única conexão aberta para múltiplas requisições, melhorando a eficiência da comunicação e reduzindo a latência.

2.3 Arquiteturas de Comunicação

As arquiteturas de comunicação são fundamentais para o desenvolvimento de sistemas distribuídos e microsserviços, proporcionando formas eficientes e estruturadas de troca de dados entre diferentes serviços. Nesta seção, falaremos sobre as duas arquiteturas que abordaremos nesse trabalho: o gRPC e o REST.

2.3.1 gRPC

O gRPC é um *framework* de comunicação remota de alto desempenho que foi inicialmente desenvolvido pelo Google. Ele é usado para conectar serviços em e entre aplicações com suporte a várias linguagens de programação, o que facilita a criação de sistemas distribuídos. Em vez de usar uma arquitetura com um único *endpoint* como o REST, o gRPC baseia-se no uso de procedimentos remotos chamados (*Remote Procedure Calls*, RPC), onde cada função ou método que um serviço deseja expor é definido explicitamente (GRPC, 2023).

O gRPC utiliza o formato de serialização *Protocol Buffers* (ProtoBuf), que é conhecido por sua eficiência e capacidade de gerar código automaticamente em várias linguagens de programação. Este formato é fundamental para definir tanto a estrutura de dados quanto o serviço em arquivos `.proto`, que especificam os métodos que podem ser chamados remotamente, juntamente com seus parâmetros e tipos de retorno (GRPC, 2022).

As comunicações no gRPC ocorrem sobre o protocolo HTTP/2, que permite múltiplas requisições simultâneas em uma única conexão, otimizando assim o desempenho e reduzindo a latência. Isso é especialmente vantajoso em ambientes de microsserviços, onde o número de requisições entre serviços pode ser muito alto.

Uma característica distintiva do gRPC é a sua capacidade de suportar quatro tipos diferentes de chamadas de serviço: chamadas unárias (uma requisição, uma resposta), *streaming* do servidor (uma requisição, múltiplas respostas), *streaming* do cliente (múltiplas requisições, uma resposta), e bidirecional *streaming* (múltiplas requisições, múltiplas respostas). Essa flexibilidade permite que os desenvolvedores escolham o padrão de comunicação mais adequado para cada caso de uso (GRPC, 2022).

Para iniciar uma comunicação usando gRPC, o cliente e o servidor devem definir e concordar com os métodos disponíveis, seus parâmetros e os tipos de retorno, conforme especificado nos arquivos `.proto`. A partir daí, os clientes podem fazer chamadas aos procedimentos como se fossem funções locais, mas que são executadas remotamente no servidor. Essa definição é realizada utilizando *Protocol Buffers*, também conhecidos como Protobuf, uma linguagem de definição de dados desenvolvida pelo Google para serializar dados estruturados. Protobuf é utilizado para comunicação em sistemas de microsserviços e

APIs, particularmente quando combinado com gRPC.

O Protobuf permite que os desenvolvedores definam suas estruturas de dados (como mensagens) e serviços em arquivos `.proto`, que são eficazes tanto para a leitura humana quanto para interpretação mecânica. A definição no arquivo `.proto` inclui tipos que podem ser primitivos (como `int32`, `float`, `double`, `bool`, `string`) ou complexos (como enumeradores e outras mensagens) (BUFFERS, 2024a).

Por exemplo, a Figura 1 exemplifica a definição de uma mensagem de um livro em um arquivo `.proto` que poderia ser usada em uma aplicação de biblioteca.

Figura 1 – Definição do objeto Livro no *Protocol Buffers*



```
message Livro {  
    string id = 1;  
    string titulo = 2;  
    string autor = 3;  
    string isbn = 4;  
    string descricao = 5;  
    string preco = 6;  
}
```

Fonte: O autor.

Neste exemplo, cada campo da mensagem Livro é atribuído um número único, que é usado no processo de serialização binária. Isso torna o *Protocol Buffers* altamente eficiente tanto em termos de espaço quanto de velocidade de processamento, pois apenas valores essenciais ou que diferem dos padrões são transmitidos ou armazenados.

A utilização do Protobuf também estende-se à definição de serviços, que especificam métodos que podem ser chamados como mostrado na Figura 2.

Figura 2 – Definição do *service* Biblioteca no *Protocol Buffers*.



```
service Biblioteca {  
    rpc GetLivro (Request) returns (Livro);  
}
```

Fonte: O autor.

No exemplo de serviço acima, `GetLivro` é um método que espera uma requisição e

retorna uma resposta do tipo Livro. Este padrão é fundamental para a implementação de APIs gRPC, onde serviços e dados são claramente definidos e estruturados para comunicações eficientes entre diferentes sistemas e linguagens de programação.

Além disso, o Protobuf suporta várias linguagens de programação, incluindo C++, Java, Python, e Go, por meio da geração automática de código-fonte a partir dos arquivos `.proto`. Isso facilita o desenvolvimento e manutenção de aplicações distribuídas, permitindo aos desenvolvedores concentrarem-se na lógica de negócios em vez de preocupações com detalhes da comunicação entre serviços ([BUFFERS, 2024b](#)).

2.3.2 REST

REST (*Representational State Transfer*) é um estilo arquitetônico de desenvolvimento de APIs web que foi descrito por Roy Fielding em sua tese de doutorado em 2000. Ele é amplamente utilizado para a construção de serviços web devido à sua simplicidade e compatibilidade com a infraestrutura da web existente. Em REST, as interações entre clientes e serviços são construídas em torno do conceito de recursos, cada um identificado por URLs ([FIELDING, 2000](#)).

O REST utiliza métodos HTTP padrão, como GET, POST, PUT e DELETE, para facilitar a leitura, criação, atualização e exclusão de recursos, respectivamente. Essa abordagem padronizada promove uma interação intuitiva com as APIs, tornando-as facilmente compreensíveis e utilizáveis por desenvolvedores.

Os dados em uma API REST são tipicamente transmitidos em formatos como JSON, facilitando a integração com diversas linguagens de programação e plataformas. A independência de formato de dados permite que os desenvolvedores escolham o mais adequado para o contexto da aplicação, enquanto mantêm a comunicação eficaz entre diferentes sistemas.

As comunicações REST são realizadas sobre o protocolo HTTP/1.1, embora o HTTP/2 também possa ser utilizado para melhorar o desempenho através de técnicas como multiplexação e compressão de cabeçalhos. Este modelo de comunicação é baseado em solicitações e respostas padrão, onde cada interação é independente, mantendo o estado do cliente entre as chamadas externamente, conhecido como *stateless*.

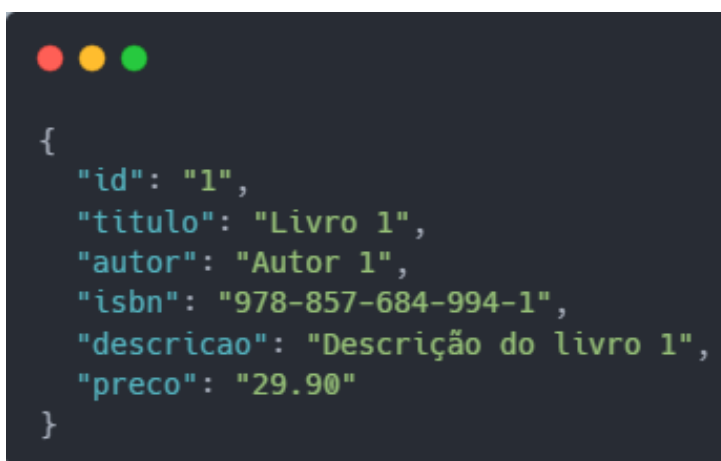
Uma característica marcante do REST é a sua capacidade de utilizar a cacheabilidade, onde as respostas podem ser armazenadas em cache para melhorar o desempenho e reduzir a carga nos servidores. Além disso, REST é flexível o suficiente para suportar comunicações síncronas e assíncronas, adaptando-se bem a diferentes necessidades de escalabilidade e interação entre serviços.

Para implementar uma API usando REST, os desenvolvedores devem definir os recursos disponíveis, seus métodos HTTP associados e as estruturas de dados para

solicitações e respostas, geralmente especificados em documentações como OpenAPI. Isso facilita a integração e a automação no uso da API, permitindo que clientes interajam com os recursos como se estivessem interagindo diretamente com objetos locais. Um dos formatos mais comuns para troca de dados em APIs REST é o JSON (*JavaScript Object Notation*), um formato leve, fácil de ler e escrever, e simples de analisar e gerar pelo computador. Desenvolvido originalmente como parte da linguagem JavaScript, JSON é hoje amplamente utilizado em diversas linguagens de programação devido à sua simplicidade e eficiência na web e em aplicações móveis.

O JSON permite que desenvolvedores estruturem dados em formatos que são naturalmente compreendidos por linguagens de programação, como objetos e *arrays*. Um exemplo típico de JSON para representar dados de um livro em uma aplicação de biblioteca digital pode ser visto na Figura 3.

Figura 3 – Definição do *payload* livro em JSON.



```
{
  "id": "1",
  "titulo": "Livro 1",
  "autor": "Autor 1",
  "isbn": "978-857-684-994-1",
  "descricao": "Descrição do livro 1",
  "preco": "29.90"
}
```

Fonte: O autor.

Neste exemplo, cada propriedade do objeto JSON corresponde a um atributo do livro, facilitando a manipulação e acesso aos dados em aplicações que consomem ou produzem esses dados. JSON é especialmente valorizado por sua interoperabilidade com a maioria das linguagens de programação modernas, o que permite que dados sejam facilmente enviados e recebidos através de APIs da web.

A utilização do JSON estende-se também à configuração e definição de estruturas mais complexas, como *arrays* de objetos e aninhamento de dados. Isso é particularmente útil em aplicações que necessitam de representar estruturas de dados hierárquicas ou listas de entidades, como mostrado na Figura 4.

Além disso, o JSON é comumente usado em comunicações na web devido à sua eficiência em transmissões de dados através de redes, sendo suportado nativamente por navegadores e servidores web. A natureza *text-based* do JSON, juntamente com sua eficiência e capacidade de ser facilmente lido e interpretado tanto por humanos quanto

Figura 4 – Exemplo de objeto JSON representando uma biblioteca



```
"biblioteca": {  
  "livros": [  
    {  
      "id": "1",  
      "titulo": "Livro 1",  
      "autor": "Autor 1"  
    },  
    {  
      "id": "2",  
      "titulo": "Livro 2",  
      "autor": "Autor 2"  
    }  
  ]  
}
```

Fonte: O autor.

por programas, faz dele um dos formatos de dados mais populares para APIs REST e configurações de *software* (JSON, 2024).

2.4 Ferramentas para os Testes de Desempenho

Esta seção descreve as ferramentas utilizadas para avaliar o desempenho das APIs REST e gRPC. Utilizamos o Apache JMeter para realizar testes de carga e Docker para criação de contêiner e coleta de métricas de CPU e Memória RAM.

2.4.1 JMeter

JMeter é um *software* de código aberto desenvolvido pela Apache *software* Foundation, projetado para realizar testes de carga, testes funcionais e medir o desempenho de aplicações. Originalmente criado por Stefano Mazzocchi para testar o desempenho do Apache JServ (agora Apache Tomcat), o JMeter evoluiu para suportar uma variedade de cenários de teste além de apenas aplicações web (HAMILTON, 2024).

O JMeter é construído em Java e funciona como uma aplicação *desktop* (JMETER, 2024). A arquitetura do JMeter é baseada em uma série de componentes que trabalham juntos para realizar testes:

- **Thread Groups:** São usados para definir o número de usuários virtuais (*threads*) que irão simular o comportamento dos usuários reais.
- **Samplers:** São responsáveis por enviar requisições aos servidores e coletar respostas.
- **Listeners:** Capturam e armazenam os resultados dos testes, permitindo a análise dos dados.
- **Timers:** Adicionam pausas entre as requisições para simular o comportamento real dos usuários.
- **Assertions:** Verificam se as respostas dos servidores estão corretas.
- **Config Elements:** Fornecem configurações adicionais para os *Samplers* e outras partes do teste.

2.4.2 Docker

Docker é uma plataforma *open source* que facilita o desenvolvimento, envio e execução de aplicações dentro de contêineres. Utilizando contêineres, Docker permite separar as aplicações da infraestrutura, possibilitando uma entrega rápida e consistente de *software*. A arquitetura do Docker usa um modelo cliente-servidor, onde o cliente Docker comunica-se com o *daemon* Docker para construir, executar e distribuir contêineres (DOCKER, 2024).

Contêineres são uma tecnologia que permite empacotar uma aplicação junto com todas as suas dependências e configurações necessárias para executá-la de maneira consistente em qualquer ambiente. Ao contrário das máquinas virtuais, que incluem um sistema operacional completo, os contêineres compartilham o *kernel* do sistema operacional do *host*, tornando-os mais leves e eficientes em termos de uso de recursos.

2.5 Considerações finais

A arquitetura REST adota um modelo mais simples para o cliente e o servidor, ao contrário do modelo de arquitetura do gRPC. Um dos objetivos deste estudo é também mostrar se a complexidade adicional do gRPC oferece vantagens de desempenho que justificariam a adoção dessa nova tecnologia. Para isso, serão utilizados o JMeter para configurar e testar a latência de cada solicitação ao servidor, e o Docker para monitorar métricas de CPU e memória.

Embora o gRPC ofereça funcionalidades avançadas, como a multiplexação de requisições e o uso do protocolo HTTP/2, especialmente úteis em cenários de alta demanda e comunicação intensiva entre microsserviços, optamos por utilizar HTTP/1.1 no modelo

REST para nossa implementação. Essa escolha se justifica pela ampla adoção do HTTP/1.1, especialmente em sistemas legados, o que permite uma comparação mais direta e realista das diferenças arquiteturais ([GROUND, 2024](#)). No capítulo seguinte, serão descritos os passos para a realização dos testes de desempenho.

3 Metodologia Proposta

Este capítulo detalha a metodologia usada para comparar o desempenho de duas APIs distintas: uma implementada usando o protocolo REST com o *microframework* Flask, e outra utilizando o protocolo gRPC com suporte da biblioteca gRPC para Python. Ambas as APIs operam sobre um banco de dados simulado contido em um arquivo JSON, que representa uma coleção de livros, cada um descrito por campos como identificador, título, autor, ISBN, descrição, e preço. Este capítulo também introduz a ferramenta de teste de carga JMeter, que é empregada para simular requisições e medir a performance das APIs sob diferentes cenários de uso.

A metodologia utilizada neste trabalho foi adaptada do estudo realizado por Bittencourt (2021) ([BITTENCOURT, 2021](#)), que conduziu uma análise comparativa entre APIs RESTful e gRPC. As adaptações incluíram a personalização dos cenários de teste, a utilização de um banco de dados em formato JSON. Essas modificações certificam uma comparação justa e objetiva entre as duas tecnologias de APIs.

3.1 Definição e Elaboração do Banco de Dados

O banco de dados tem um papel fundamental nos testes de performance das APIs REST e gRPC, uma vez que ambas acessarão os mesmos dados. É essencial que o banco de dados seja estruturado de maneira a não favorecer nenhuma das tecnologias em teste. Para isso, foi elaborado um banco de dados simulado usando um arquivo JSON, que contém entradas representando livros, simulando um sistema de gestão de biblioteca. Este arquivo contém múltiplas entradas (livros).

3.1.1 Estrutura do Banco de Dados

O arquivo `books.json`, esquematizado pela Figura 5, serve como o banco de dados para este teste, em que cada entrada é composta dos seguintes campos:

- **ID:** Identificador único de cada livro, usado para referenciar e acessar os livros especificamente.
- *Author:* Autor do livro.
- **ISBN:** Número padrão internacional de identificação de livro.
- *Description:* Uma descrição detalhada, proporcionando contexto e informações adicionais sobre o livro.

- *Price*: Preço do livro.

Figura 5 – Estrutura do banco de dados que será consumido por ambas as APIs

book	
id	string
title	string
author	string
isbn	string
description	string
price	string

Fonte: O autor.

3.2 Definição dos Testes no JMeter

Para a realização dos testes de desempenho das APIs REST e gRPC, foram configurados dois grupos de *threads* (*Thread Groups*) no JMeter: um dedicado à API REST e outro à API gRPC. O objetivo principal destes testes é medir o desempenho de ambas as APIs sob diferentes condições de carga.

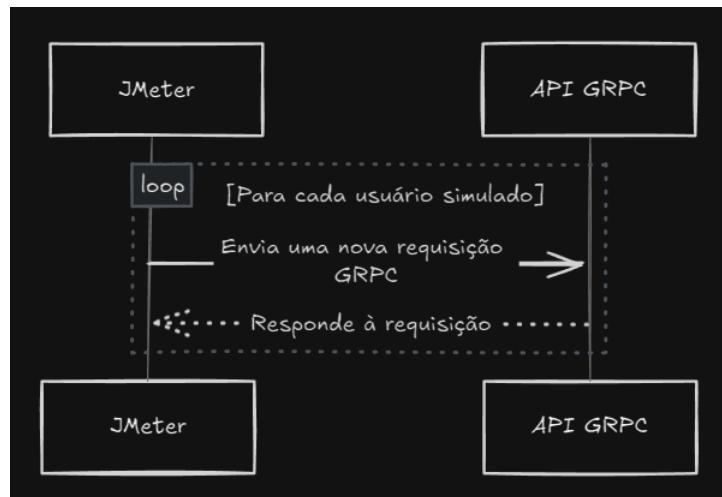
Nos testes foram utilizados dois *plugins* específicos do JMeter: o plugin pg@gc - *Response Time Over Time* e um *plugin* específico para gRPC. O *plugin* pg@gc - *Response Time Over Time* (PLUGINS, 2024), foi utilizado para medir e visualizar o tempo de resposta das APIs ao longo do tempo. O *plugin* para gRPC (OSS, 2024), foi necessário para coletar as métricas de desempenho da API gRPC visto que o gRPC ainda não suporta nativamente essa arquitetura.

A configuração padrão do JMeter foi mantida, com exceção da adição das informações necessárias para realizar as requisições às APIs. Não foi utilizado *ramp-up*, o que significa que todos os usuários iniciaram simultaneamente e continuaram até o fim do teste. Todas as configurações referente aos testes estão disponíveis no formato XML dentro da pasta testes no repositório (BECKHAUSER, 2024).

3.2.1 Cenário de Teste gRPC

A Figura 7 ilustra o fluxo de requisições e respostas no cenário de teste para a API gRPC. Neste cenário, o JMeter simula usuários que enviam requisições gRPC à API. Para cada usuário simulado, uma nova requisição gRPC é enviada e uma resposta é recebida, permitindo medir o desempenho da API sob carga.

Figura 6 – Fluxo de Requisição e Resposta da API gRPC no JMeter



Fonte: O autor.

3.2.2 Cenário de Teste REST

A Figura 7 mostra o fluxo de requisições e respostas no cenário de teste para a API REST. Similarmente ao cenário gRPC, o JMeter simula usuários que enviam requisições HTTP à API REST. Para cada usuário simulado, uma nova requisição HTTP é enviada e uma resposta é recebida, permitindo avaliar o desempenho da API REST sob diferentes condições de carga.

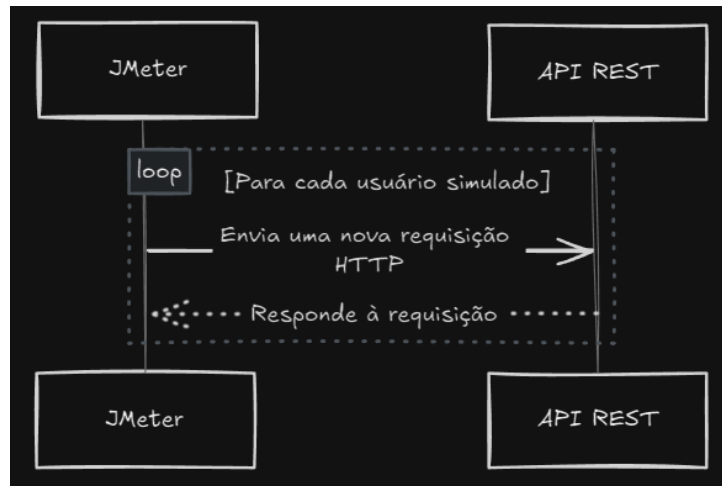
3.3 Definição de Captura das Métricas

As métricas de desempenho das APIs REST e gRPC foram capturadas utilizando o comando do Docker ‘docker stats’, o qual fornece estatísticas em tempo real de uso de recursos para os contêineres Docker em execução. Esta seção detalha o processo de captura das métricas e os parâmetros observados.

3.3.1 Comando Docker Utilizado

O comando ‘docker stats’ foi utilizado da seguinte forma para monitorar o contêiner que hospeda a API em teste:

Figura 7 – Fluxo de Requisição e Resposta da API REST no JMeter



Fonte: O autor.

```
docker stats <ID_CONTAINER>
```

Onde ‘<ID_CONTAINER>’ é o ID do contêiner específico que está sendo monitorado.

3.3.2 Parâmetros Monitorados

As seguintes métricas foram monitoradas e registradas durante a execução dos testes de carga com o JMeter:

- **Uso de CPU (%):** Percentual de utilização da CPU pelo contêiner, fornecendo uma indicação de quanta carga a API está impondo ao processador do sistema.
- **Uso de Memória (MB):** Quantidade de memória RAM sendo utilizada pelo contêiner. Esta métrica é crucial para entender o consumo de recursos de memória pelas APIs durante a execução de operações de leitura e escrita.

3.3.3 Procedimento de Captura de Métricas

O procedimento para capturar as métricas envolveu os seguintes passos:

1. **Configuração Inicial:** Iniciar os contêineres Docker que contém a API REST ou gRPC.
2. **Execução dos Testes:** Utilizar o JMeter para simular as requisições conforme os cenários de teste definidos (Obtenção de Um Registro, Obtenção de Dez Registros, Inserção de Um Registro e Inserção de Dez Registros).

3. **Monitoramento em Tempo Real:** Executar o comando ‘docker stats’ para monitorar em tempo real o uso de recursos pelo contêiner durante a execução dos testes.
4. **Registro das Métricas:** Registrar os valores das métricas monitoradas em intervalos regulares, garantindo que todos os aspectos do desempenho das APIs sejam capturados e analisados.

3.4 API REST

Para este projeto, foi desenvolvida uma API REST utilizando Python com o *microframework* Flask, projetada para interagir com o banco de dados em formato JSON detalhado anteriormente. Esta API facilita a manipulação dos dados de livros armazenados e é composta por diversos *endpoints* que permitem realizar operações CRUD (*Create, Read, Update, Delete*) de forma eficiente. O código-fonte completo está disponível no seguinte repositório do GitHub ([BECKHAUSER, 2024](#)).

3.4.1 Estrutura da API REST

A API possui uma URL base que será usada para acessar os recursos relacionados aos livros. Os *endpoints* específicos são configurados como segue:

3.4.1.1 *Endpoint* para buscar todos os livros disponíveis

URL: /books

Método: GET

Retorno: Este *endpoint* retorna uma lista de todos os livros disponíveis no banco de dados, onde cada livro é representado como um objeto JSON com todos os seus campos: ID, Título, Autor, ISBN, Descrição e Preço.

3.4.1.2 *Endpoint* para buscar um livro específico pelo ID

URL: /books/<string:book-id>

Método: GET

Retorno: Este *endpoint* busca um livro específico pelo ID fornecido na URL. Se encontrado, retorna um objeto JSON contendo detalhes desse livro. Caso contrário, retorna uma mensagem de erro indicando que o livro não foi encontrado.

3.4.1.3 *Endpoint* para adicionar um novo livro

URL: /books

Método: POST

Corpo da Requisição: JSON contendo os detalhes do novo livro (título, autor, ISBN, descrição, preço).

Retorno: Retorna um objeto JSON com os detalhes do livro adicionado.

3.4.1.4 *Endpoint* para adicionar uma lista de livros

URL: /books/list

Método: POST

Corpo da Requisição: JSON contendo uma lista de objetos livro com os detalhes dos novos livros (título, autor, ISBN, descrição, preço).

Retorno: Retorna um objeto JSON com uma lista de detalhes dos livros adicionados, incluindo seus novos IDs gerados automaticamente.

3.5 API gRPC

Para este projeto, foi desenvolvida uma API utilizando o *framework* gRPC, projetada para interagir com o banco de dados JSON previamente descrito. A API gRPC foi implementada em Python, utilizando as bibliotecas gRPC Python fornecidas pelo Google. Todas as chamadas gRPC utilizadas neste trabalho seguem o **modelo unário**, onde o cliente envia uma única solicitação ao servidor e recebe uma única resposta em retorno, semelhante ao funcionamento de uma chamada HTTP tradicional. O código-fonte completo está disponível no seguinte repositório do GitHub ([BECKHAUSER, 2024](#)).

3.5.1 Definição dos Serviços gRPC

A API gRPC utiliza *Protocol Buffers* (protobufs) para definir a estrutura dos dados e os métodos disponíveis. Abaixo estão os serviços definidos para manipulação dos dados dos livros, na Figura 8 podemos ver a estrutura do objeto *Book* e na Figura 9 podemos ver a definição dos *services* dentro do *Protocol Buffers*.

3.5.1.1 *Service* para recuperar todos os livros (GetAll)

Método RPC: GetAll (definição mostrada na Figura 10)

Método: GetAllRequest (vazio pois não requer parâmetros)

Retorno: Retorna uma lista de livros

3.5.1.2 *Service* para recuperar um livro específico pelo ID (GetOne):

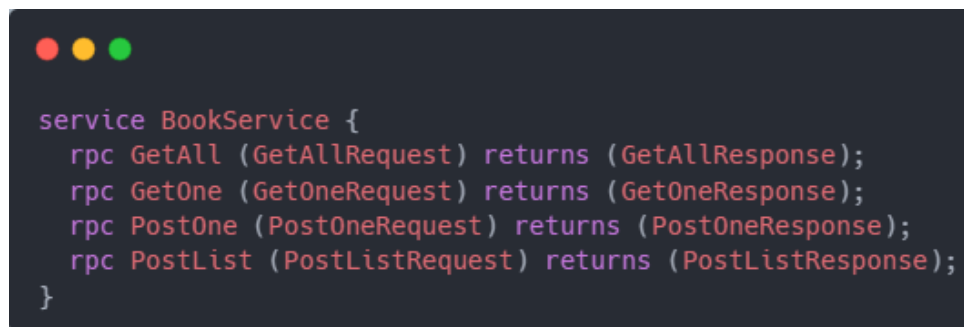
Método RPC: GetOne (definição mostrada na Figura 11)

Método: GetOneRequest que inclui o id do livro

Figura 8 – Definição de Livro no *Protocol Buffers*

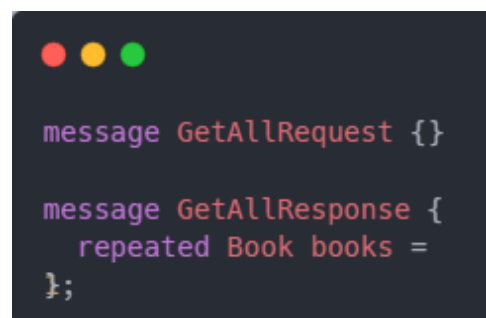
```
message Book {  
    string id = 1;  
    string title = 2;  
    string author = 3;  
    string isbn = 4;  
    string description = 5;  
    string price = 6;  
}
```

Fonte: O autor.

Figura 9 – Definição dos *services* no *Protocol Buffers*

```
service BookService {  
    rpc GetAll (GetAllRequest) returns (GetAllResponse);  
    rpc GetOne (GetOneRequest) returns (GetOneResponse);  
    rpc PostOne (PostOneRequest) returns (PostOneResponse);  
    rpc PostList (PostListRequest) returns (PostListResponse);  
}
```

Fonte: O autor.

Figura 10 – Definição do método GetAll dentro no *Protocol Buffers*

```
message GetAllRequest {}  
  
message GetAllResponse {  
    repeated Book books =  
};
```

Fonte: O autor.

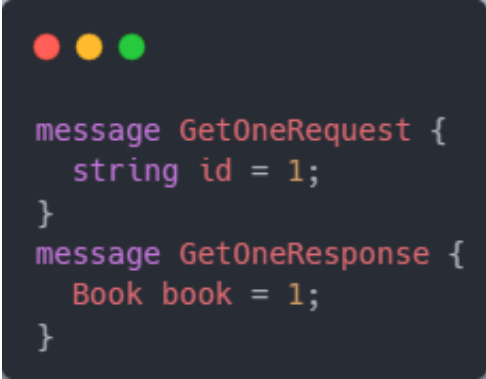
Retorno: Retorna o livro correspondente

3.5.1.3 Service para adicionar um novo livro (PostOne)

Método RPC: PostOne (definição mostrada na Figura 12)

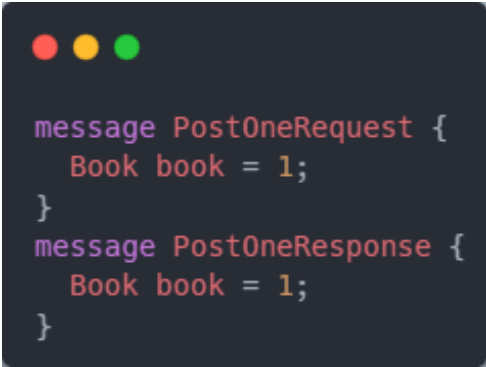
Método: PostOneRequest que inclui os detalhes do livro a ser adicionado

Retorno: Retorna o livro adicionado

Figura 11 – Definição do método GetOne dentro no *Protocol Buffers*

```
message GetOneRequest {  
    string id = 1;  
}  
message GetOneResponse {  
    Book book = 1;  
}
```

Fonte: O autor.

Figura 12 – Definição do método PostOne no *Protocol Buffers*

```
message PostOneRequest {  
    Book book = 1;  
}  
message PostOneResponse {  
    Book book = 1;  
}
```

Fonte: O autor.

3.5.1.4 Service para adicionar uma lista de novos livro (PostList)

Método RPC: PostList (definição mostrada na Figura 13)

Método: PostListRequest que inclui os detalhes dos livros a serem adicionado

Retorno: Retorna os livros adicionados

Figura 13 – Definição do método PostOne no *Protocol Buffers*

```
message PostListRequest {  
    repeated Book books =  
};  
message PostListResponse {  
    repeated Book books =  
};
```

Fonte: O autor.

3.6 Configuração dos Testes

Este trabalho propõe responder a três perguntas diferentes. A descrição detalhada de cada cenário é apresentada nesta subseção, com foco em entender e comparar o desempenho das APIs REST e gRPC sob condições controladas de teste.

3.6.1 Teste 1: Obtenção de Um Registro

O primeiro cenário de teste tem como objetivo verificar e comparar o comportamento das APIs REST e gRPC quando um *payload* pequeno é requisitado. Esse teste é crucial para entender como cada API maneja requisições leves, que são comuns em muitos cenários de uso real, especialmente em dispositivos móveis e aplicações que requerem alta eficiência em redes com baixa largura de banda.

3.6.1.1 Configuração do Teste

O teste é configurado para que as duas APIs façam o mesmo tipo de consulta ao banco de dados com o menor volume de dados possível. Isso implica utilizar um método que retorne o mínimo de informações. O objetivo é minimizar o tempo de processamento das requisições ao banco de dados, evitando qualquer viés que possa ser introduzido por diferenças na quantidade de dados requisitados.

O *endpoint* e *service* selecionados foram:

API REST: GET /books/book_id

API gRPC: *Service* GetOne

3.6.2 Teste 2: Obtenção de Cinco Mil Registros

O segundo cenário de teste avalia o comportamento das APIs REST e gRPC quando um *payload* grande é requisitado. Este teste é crucial para determinar a eficiência das APIs ao manusear grandes volumes de dados, simulando uma operação de busca massiva onde todos os livros são retornados com todos os campos detalhados. Este cenário é representativo de operações de *data-fetching* em grande escala, comum em análises de dados e situações onde o cliente precisa de acesso completo ao conjunto de dados.

3.6.2.1 Configuração do Teste

O teste é configurado para que ambas as APIs executem operações de consulta que retornam grandes quantidades de dados. Para garantir justiça e comparabilidade direta, ambas as APIs são configuradas para acessar e transmitir o mesmo volume de dados, consistindo na recuperação de todos os livros disponíveis no banco de dados com todos os seus campos.

Endpoints e Serviços Seleccionados:

API REST: GET /books

API gRPC: *Service* GetAllRequest

3.6.3 Teste 3: Inserção de Um Registro

Este cenário de teste visa avaliar a eficiência e agilidade das APIs REST e gRPC na inserção de dados com *payloads* pequenos. É importante para entender como cada tecnologia lida com operações de escrita leves, comuns em aplicações que realizam inserções frequentes de dados em pequenas quantidades.

3.6.3.1 Configuração do Teste:

O teste é projetado para simular a inserção de um único item de dados, para representar um *payload* pequeno. O objetivo é minimizar a complexidade e focar na capacidade das APIs de processar rapidamente novas entradas de dados.

Endpoints e Serviços Seleccionados:

API REST: POST /books (exemplo mostrado na Figura 14)

Figura 14 – Exemplo de requisição para o *endpoint* POST /books

A imagem mostra um exemplo de requisição JSON para o endpoint POST /books. O código está em um editor de texto com fundo escuro e cores de sintaxe. O JSON contém os campos: title, author, isbn, description e price, todos com valores fictícios.

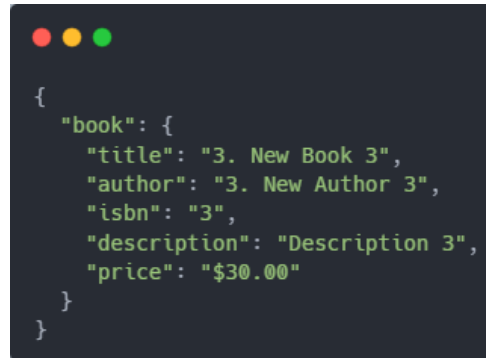
```
{
  "title": "3. New Book 3",
  "author": "3. New Author 3",
  "isbn": "3",
  "description": "Description 3",
  "price": "$30.00"
}
```

Fonte: O autor.

API gRPC: *Service* PostOne (exemplo mostrado na Figura 15)

3.6.4 Teste 4: Inserção de Dez Registros

O objetivo deste teste é avaliar a eficácia das APIs REST e gRPC na gestão de inserções de dados com *payloads* grandes. Este cenário é crítico para entender como as tecnologias se comportam ao processar grandes volumes de dados de uma só vez, como seria o caso em aplicações de análise de dados em massa, migrações de dados, ou sincronizações de grandes registros.

Figura 15 – Exemplo de requisição para o *service* PostOne

```
{
  "book": {
    "title": "3. New Book 3",
    "author": "3. New Author 3",
    "isbn": "3",
    "description": "Description 3",
    "price": "$30.00"
  }
}
```

Fonte: O autor.

3.6.4.1 Configuração do Teste:

O teste simula a inserção de múltiplos itens, cada um contendo vários campos detalhados, para representar um *payload* grande. Este teste é importante para identificar possíveis limitações de performance ou gargalos em ambientes de produção que requerem o processamento de grandes quantidades de dados inseridos simultaneamente.

Endpoints e Serviços Seleccionados:

API REST : POST /books/list (exemplo mostrado na Figura 16)

Figura 16 – Exemplo de requisição para o *endpoint* POST /books/list

```
[
  {
    "title": "4. New Book 4",
    "author": "4. New Author 4",
    "isbn": "4",
    "description": "Description 4",
    "price": "$40.00"
  },
  {
    "title": "5. New Book 5",
    "author": "5. New Author 5",
    "isbn": "5",
    "description": "Description 5",
    "price": "$50.00"
  }
]
```

Fonte: O autor.

API gRPC: *Service* PostList (exemplo mostrado na Figura 17)

Figura 17 – Exemplo de requisição para o *service* PostList

```
{
  "books": [
    {
      "title": "6. New Book 6",
      "author": "6. New Author 6",
      "isbn": "6",
      "description": "Description 6",
      "price": "$60.00"
    },
    {
      "title": "7. New Book 7",
      "author": "7. New Author 7",
      "isbn": "7",
      "description": "Description 7",
      "price": "$70.00"
    }
  ]
}
```

Fonte: O autor.

3.7 Considerações Finais

A metodologia detalhada neste capítulo estabelece um processo claro para comparar o desempenho das APIs REST e gRPC. Utilizando um banco de dados simulado em formato JSON e a ferramenta JMeter para simulação de carga, foram definidos cenários específicos de testes que incluem operações de leitura e escrita com diferentes tamanhos de *payload*.

O uso do comando ‘docker stats’ para monitorar o uso de CPU e memória dos contêineres Docker permitiu uma análise precisa do consumo de recursos das APIs durante os testes. Essa abordagem proporciona uma visão abrangente da eficiência e escalabilidade de cada tecnologia.

Os resultados obtidos fornecerão uma boa direção para entender as vantagens e limitações das APIs REST e gRPC, ajudando a escolher a tecnologia mais adequada para diferentes necessidades de aplicação.

4 Resultados

Este capítulo detalha os procedimentos dos testes, os resultados obtidos e as conclusões resultantes. O objetivo principal foi atingido ao identificar as qualidades e defeitos de cada arquitetura, utilizando três métricas principais: tempo de resposta das requisições, uso de CPU e consumo de memória.

4.1 Especificação

Para a realização dos testes, foram utilizados dois contêineres Docker com a configuração de 1 núcleo de CPU podendo chegar até a 2, e 1 GB de RAM. O computador que hospeda o docker é equipado com um processador Ryzen 5 3600X, 32 GB de RAM e sistema operacional Windows 10.

Os testes buscaram comparar as arquiteturas REST e gRPC em diferentes cenários. Cada teste envolveu dois volumes de dados: no primeiro cenário, com 20 usuários, e no segundo cenário, com 100 usuários. Para ambos os cenários foram simuladas várias requisições ao longo de 60 segundos.

A Figura 18 detalha os cenários de teste utilizados, especificando a ação realizada, o número de usuários simulados e a duração de cada teste.

Figura 18 – Cenários para a Comparação das APIs REST e gRPC que serão Testados.

	Ação	Nº de Usuários	Duração do Teste
Teste 1	Obtenção de um registro	20	60 segundos
Teste 1	Obtenção de um registro	100	60 segundos
Teste 2	Obtenção de cinco mil registros	20	60 segundos
Teste 2	Obtenção de cinco mil registros	100	60 segundos
Teste 3	Inserção de um registro	20	60 segundos
Teste 3	Inserção de um registro	100	60 segundos
Teste 4	Inserção de dez registros	20	60 segundos
Teste 4	Inserção de dez registros	100	60 segundos

Fonte: O autor.

Em todos os testes, serão coletadas três variáveis que serão analisadas e comparadas: tempo de resposta das requisições, porcentagem de uso da CPU pelo servidor e porcentagem de uso de memória pelo servidor.

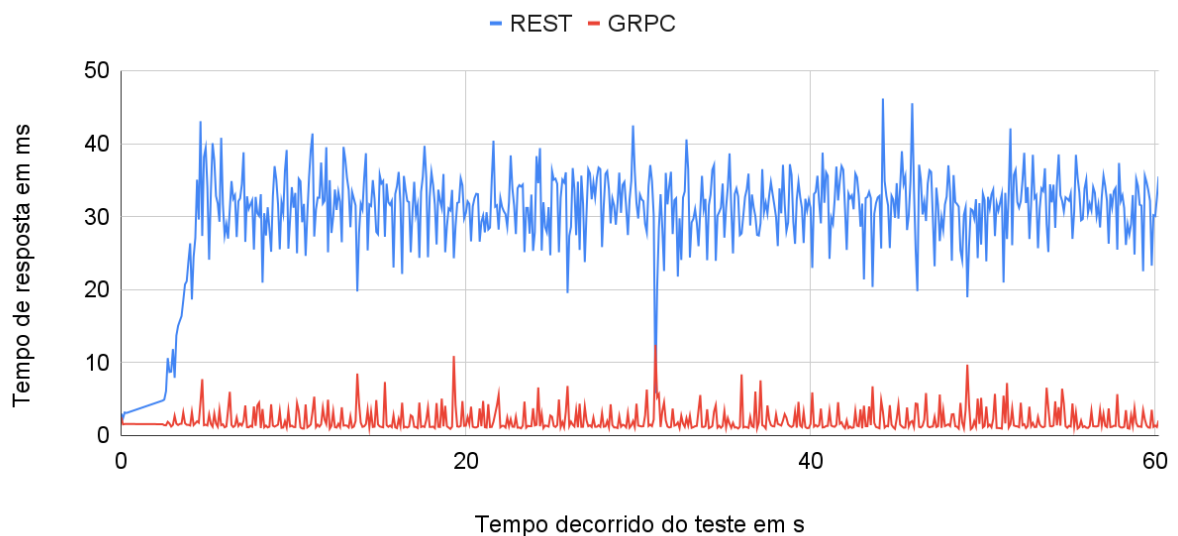
4.2 Teste 1: Obtenção de Um Registro

Para este teste, cada requisição foi configurada para buscar especificamente o registro do livro de ID 1, garantindo que ambas as APIs retornassem consistentemente o mesmo dado. Apenas um registro foi solicitado em cada requisição para manter a consistência dos resultados.

4.2.1 Amostra com 20 usuários

Para uma amostra de 20 usuários, a Figura 19 ilustra uma comparação do tempo de resposta das duas arquiteturas de comunicação. Isso indica que a arquitetura REST possui um tempo de resposta significativamente maior em comparação com a arquitetura gRPC. A média do tempo de resposta da arquitetura REST ficou em 31,84 milissegundos, enquanto que a média da arquitetura gRPC permaneceu em 1,34 milissegundos. O desvio padrão na arquitetura REST foi de 5,78 milissegundos, enquanto que na gRPC foi de 1,55 milissegundos, indicando uma maior consistência nos tempos de resposta da gRPC.

Figura 19 – Tempo de resposta para teste com obtenção de um registro em amostra de 20 usuários.

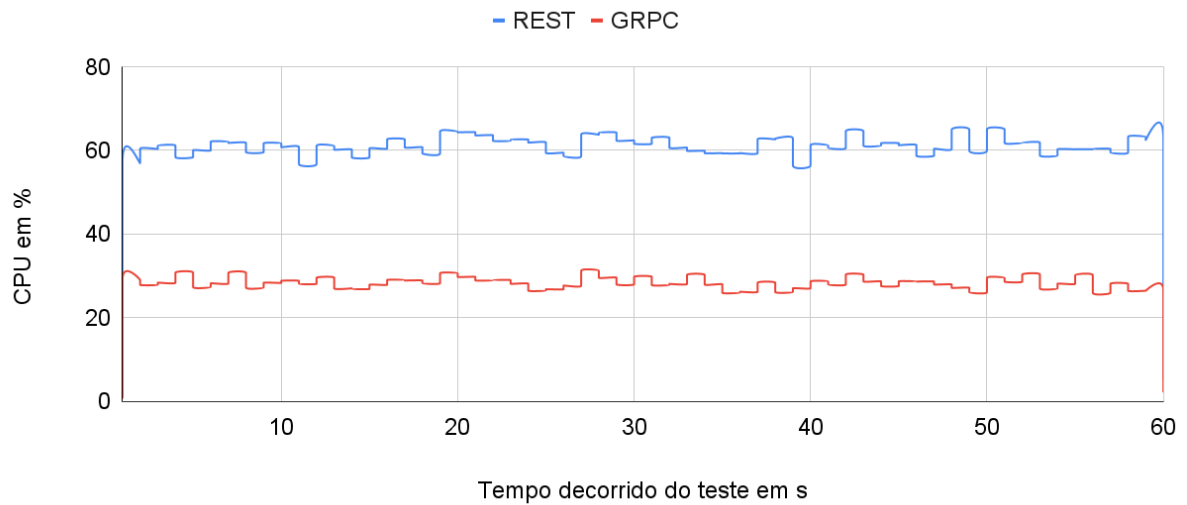


Fonte: O autor.

Percebe-se também na Figura 20 que o consumo de CPU na arquitetura gRPC é menor, com uma média de 28,11%, em comparação com os 60,91% da arquitetura REST. Isso significa que nesse teste a arquitetura gRPC possui um resultado mais eficiente.

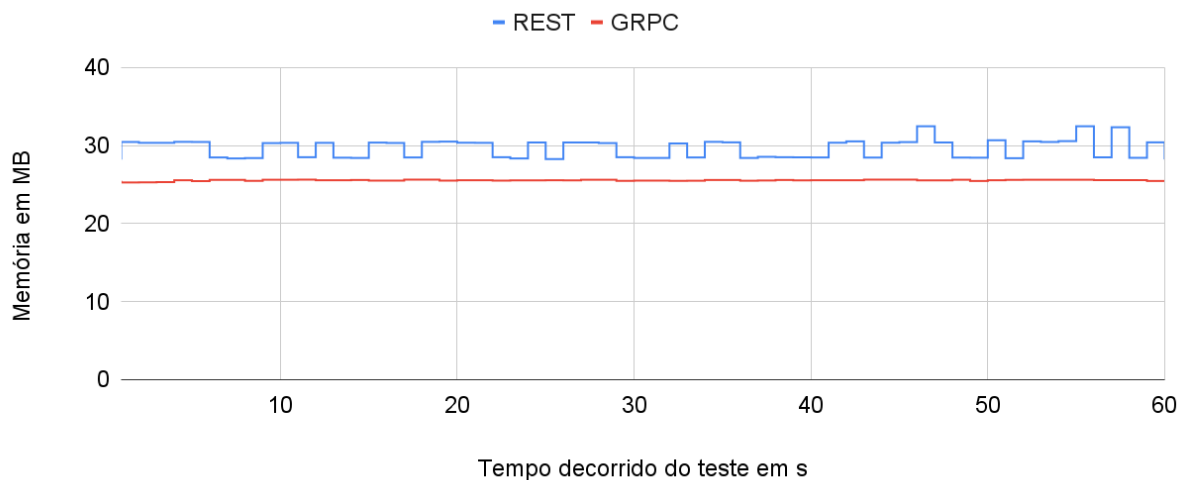
A Figura 21 apresenta uma comparação detalhada do consumo de memória entre as arquiteturas REST e gRPC. Em termos de consumo de memória, a arquitetura REST apresentou uma média de 30,34MB, enquanto que a arquitetura gRPC teve uma média de 25,56MB, mostrando novamente uma vantagem para a gRPC.

Figura 20 – Consumo de CPU para teste com obtenção de um registro em amostra de 20 usuários.



Fonte: O autor.

Figura 21 – Consumo de memória para teste com obtenção de um registro em amostra de 20 usuários.



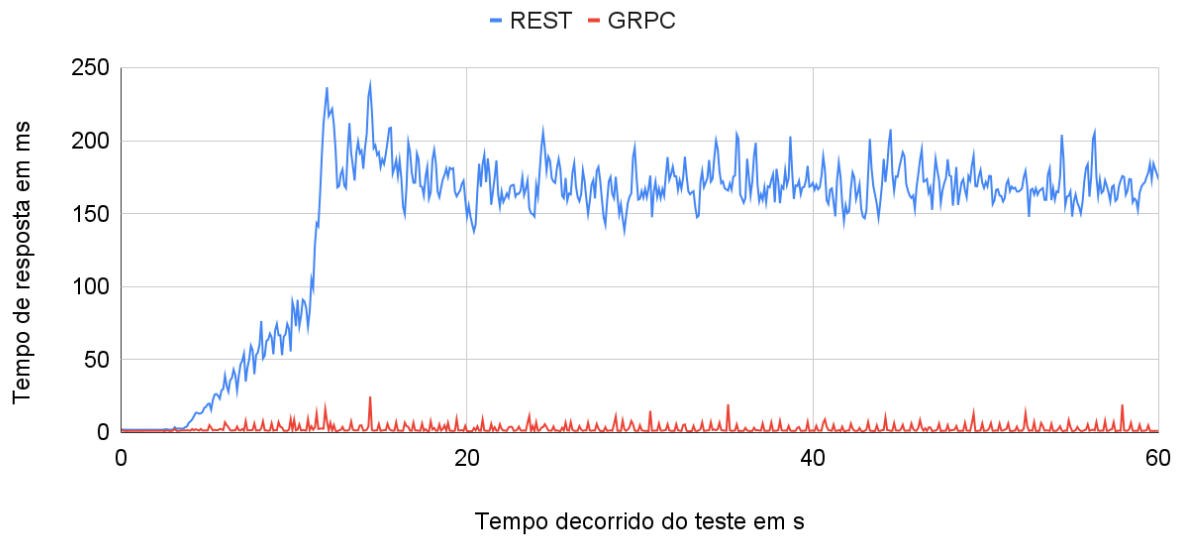
Fonte: O autor.

4.2.2 Amostra com 100 usuários

Para esse segundo teste, com 100 usuários, o objetivo é verificar se o comportamento se assemelha ao cenário anterior com mais usuários consumindo a API. Para a métrica de tempo de resposta, elaborou-se o gráfico representado pela Figura 22.

Pode-se observar que a arquitetura REST ainda possui um tempo de resposta maior do que a arquitetura gRPC, com uma média de 167,20 milissegundos para REST e 1,38 milissegundos para gRPC. O desvio padrão para REST foi de 51,75 milissegundos, enquanto que para gRPC foi de 2,68 milissegundos, novamente mostrando maior consistência na arquitetura gRPC.

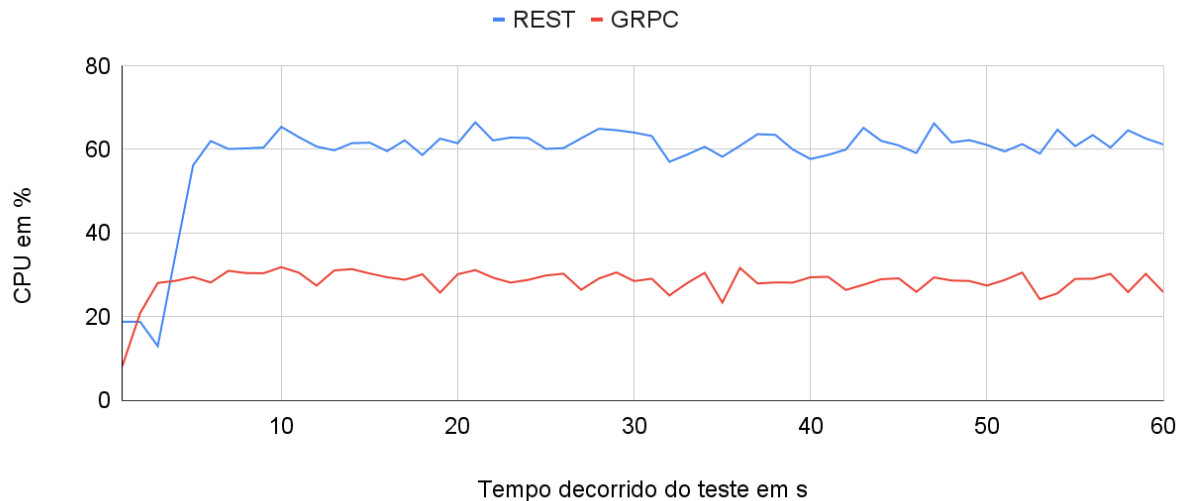
Figura 22 – Tempo de resposta para teste com obtenção de um registro em amostra de 100 usuários.



Fonte: O autor.

Na métrica de consumo de CPU, a arquitetura REST continuou a usar mais CPU, com uma média de 61,11%, em comparação com os 28,99% da arquitetura gRPC.

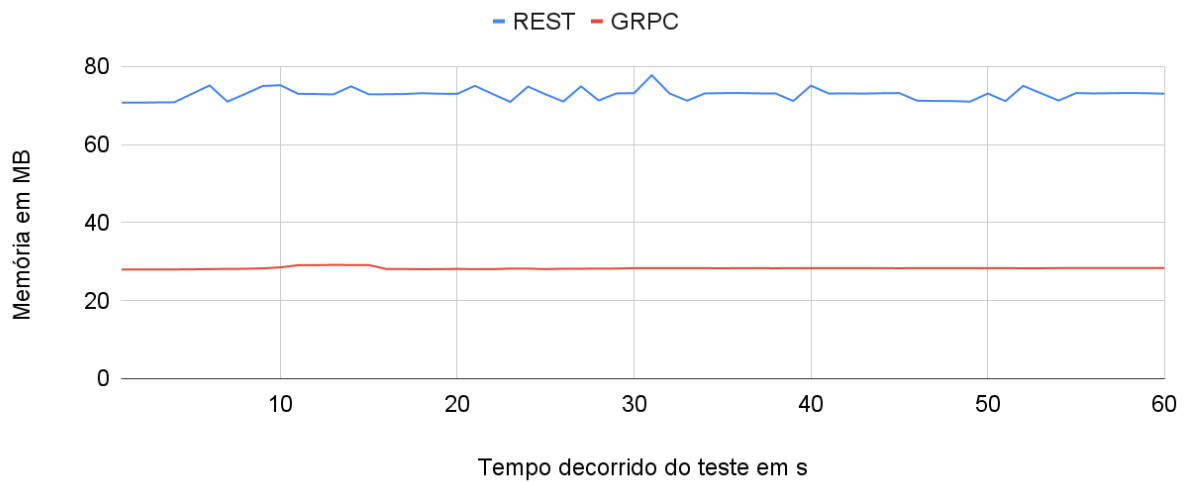
Figura 23 – Consumo de CPU para teste com obtenção de um registro em amostra de 100 usuários.



Fonte: O autor.

Finalmente, para a métrica de consumo de memória, a arquitetura REST apresentou uma média de 73,08MB, enquanto que a arquitetura gRPC teve uma média de 28,32MB, indicando uma eficiência superior da gRPC em todos os cenários testados.

Figura 24 – Consumo de memória para teste com obtenção de um registro em amostra de 100 usuários.



Fonte: O autor.

4.3 Teste 2: Obtenção de Cinco Mil Registros

Para esse teste cada requisição buscava todos os livros no banco de dados para que ambas as APIs retornassem sempre o mesmos dados.

4.3.1 Amostra com 20 usuários

Para uma amostra de 20 usuários, a Figura 25 ilustra uma comparação do tempo de resposta das duas arquiteturas de comunicação. A arquitetura REST apresentou uma média de tempo de resposta de 347,75 milissegundos, enquanto que a arquitetura gRPC teve uma média significativamente menor de 105,29 milissegundos. O desvio padrão foi de 100,29 milissegundos para REST e 74,36 milissegundos para gRPC, indicando que a gRPC não só é mais rápida, mas também mais estável em suas respostas.

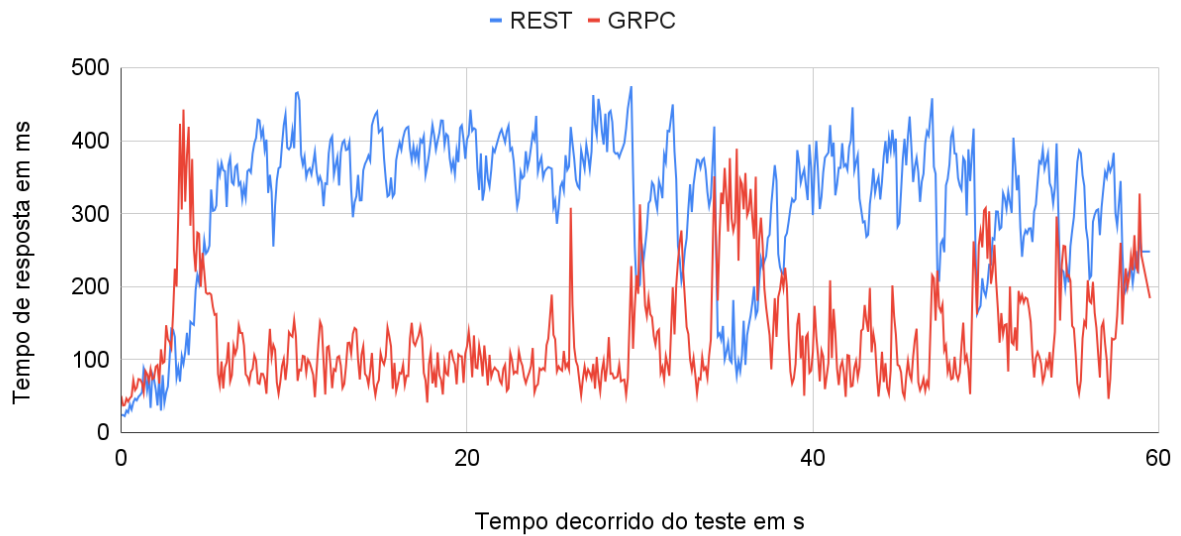
Observa-se também na Figura 26 que o consumo de CPU na arquitetura gRPC é menor, com uma média de 65,89%, em comparação com os 74,76% da arquitetura REST. Isso demonstra que a arquitetura gRPC é mais eficiente em termos de processamento.

Em termos de consumo de memória como podemos ver na Figura 27, a arquitetura REST apresentou uma média de 88,46MB, enquanto que a arquitetura gRPC teve uma média de 63,33MB. Novamente, a arquitetura gRPC mostra uma vantagem clara em termos de eficiência.

4.3.2 Amostra com 100 usuários

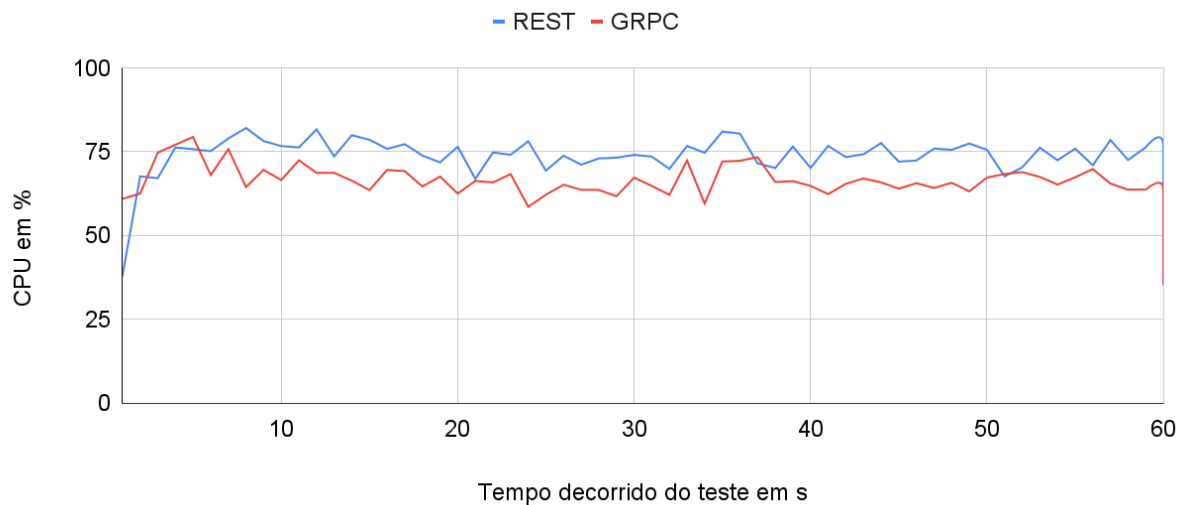
Para uma amostra de 100 usuários, a Figura 28 ilustra uma comparação do tempo de resposta das duas arquiteturas de comunicação. A arquitetura REST apresentou uma

Figura 25 – Tempo de resposta para teste com obtenção de cinco mil registros em amostra de 20 usuários.



Fonte: O autor.

Figura 26 – Consumo de CPU para teste com obtenção de cinco mil registros em amostra de 20 usuários.

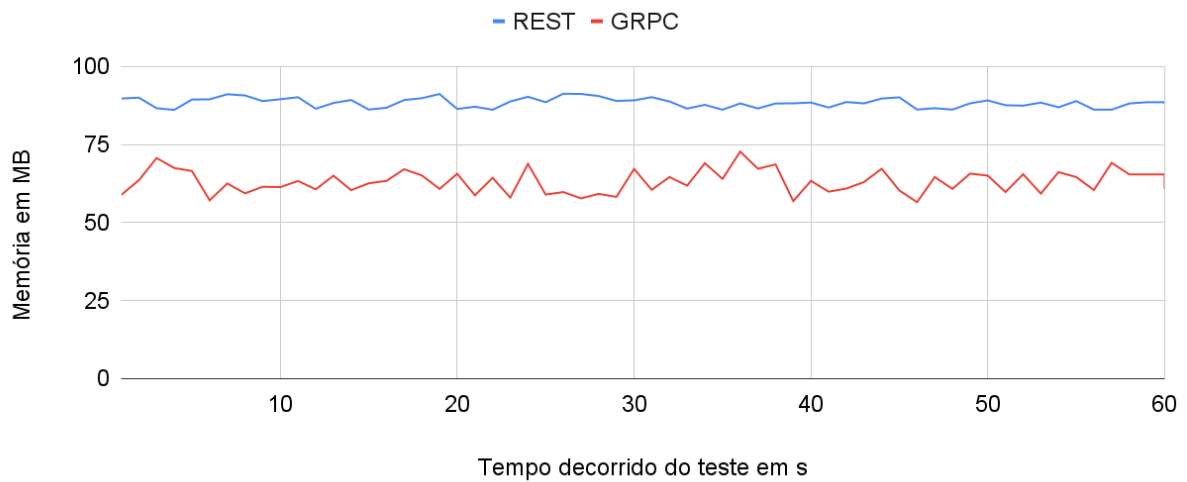


Fonte: O autor.

média de tempo de resposta de 1911,6 milissegundos, enquanto que a arquitetura gRPC teve uma média significativamente menor de 24,4 milissegundos. O desvio padrão foi de 524,66 milissegundos para REST e 162,48 milissegundos para gRPC, indicando que a gRPC é mais rápida e mantém uma maior consistência nas respostas mesmo com um maior número de usuários.

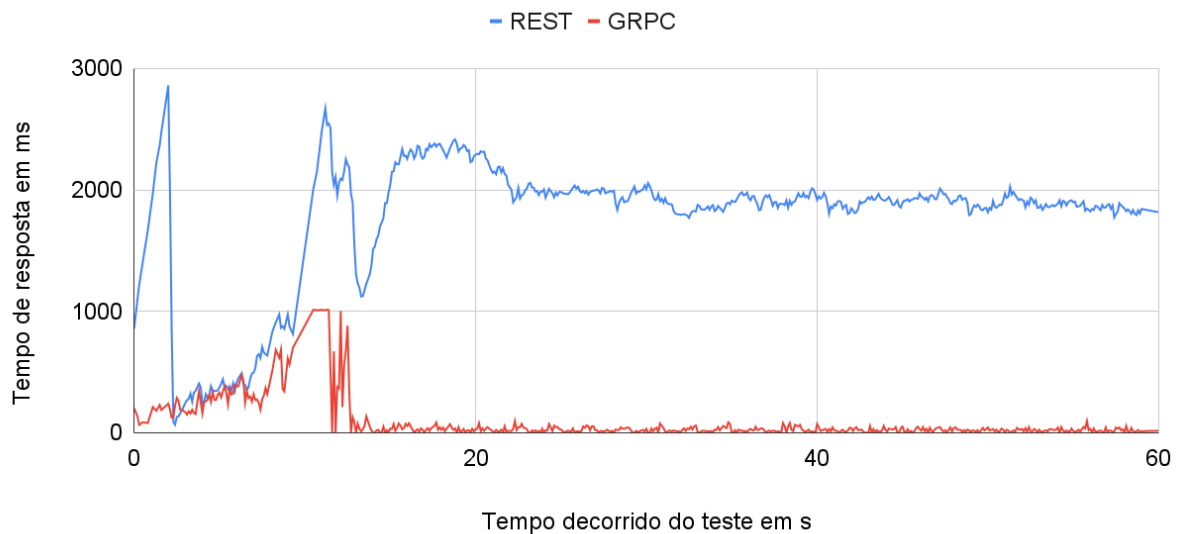
O consumo de CPU na arquitetura gRPC continuou a ser significativamente menor, com uma média de 27,43%, em comparação com os 81,93% da arquitetura REST. Isso reforça a eficiência superior da gRPC.

Figura 27 – Consumo de memória para teste com obtenção de cinco mil registros em amostra de 20 usuários.



Fonte: O autor.

Figura 28 – Tempo de resposta para teste com obtenção de cinco mil registros em amostra de 100 usuários.



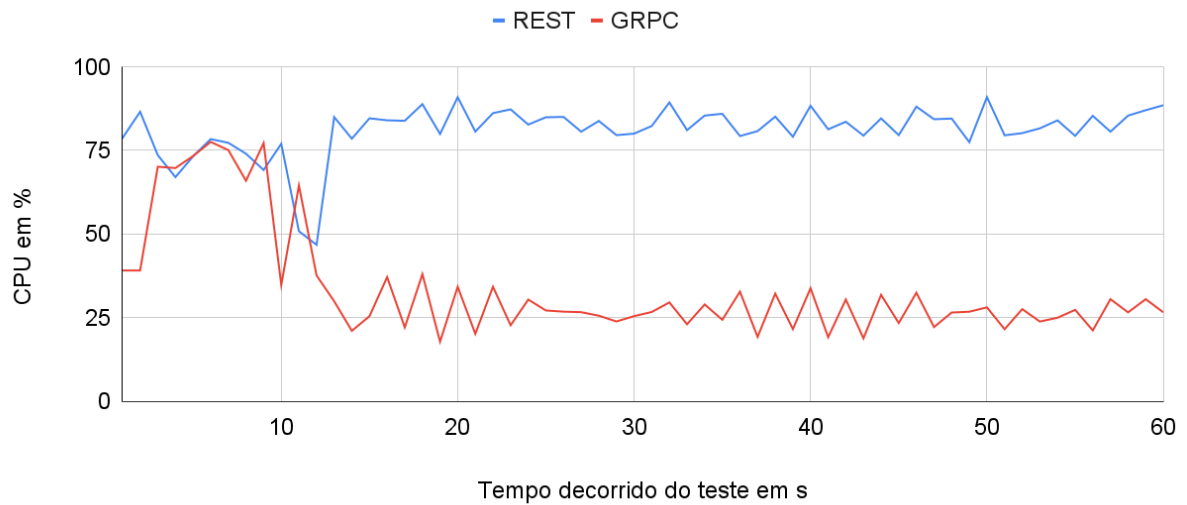
Fonte: O autor.

Finalmente, para a métrica de consumo de memória como podemos, a arquitetura REST apresentou uma média de 88,84MB, enquanto que a arquitetura gRPC teve uma média de 60,68MB, mostrando novamente uma vantagem clara para a gRPC em termos de eficiência de recursos.

4.4 Teste 3: Inserção de Um Registro

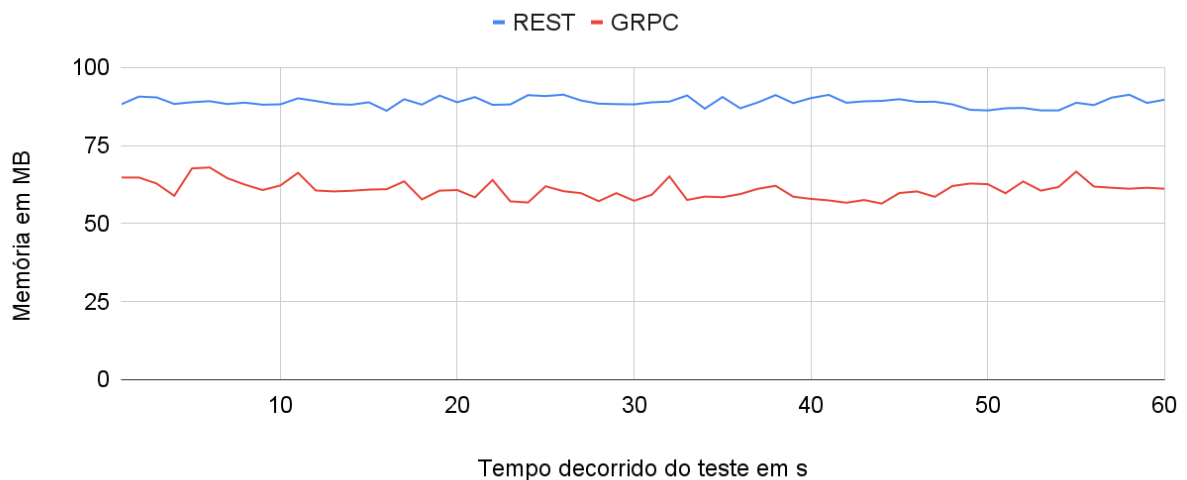
Para este teste, cada requisição inseriu um registro contendo o mesmo *payload*, garantindo assim a igualdade dos dados.

Figura 29 – Consumo de CPU para teste com obtenção de cinco mil registros em amostra de 100 usuários.



Fonte: O autor.

Figura 30 – Consumo de memória para teste com obtenção de cinco mil registros em amostra de 100 usuários.

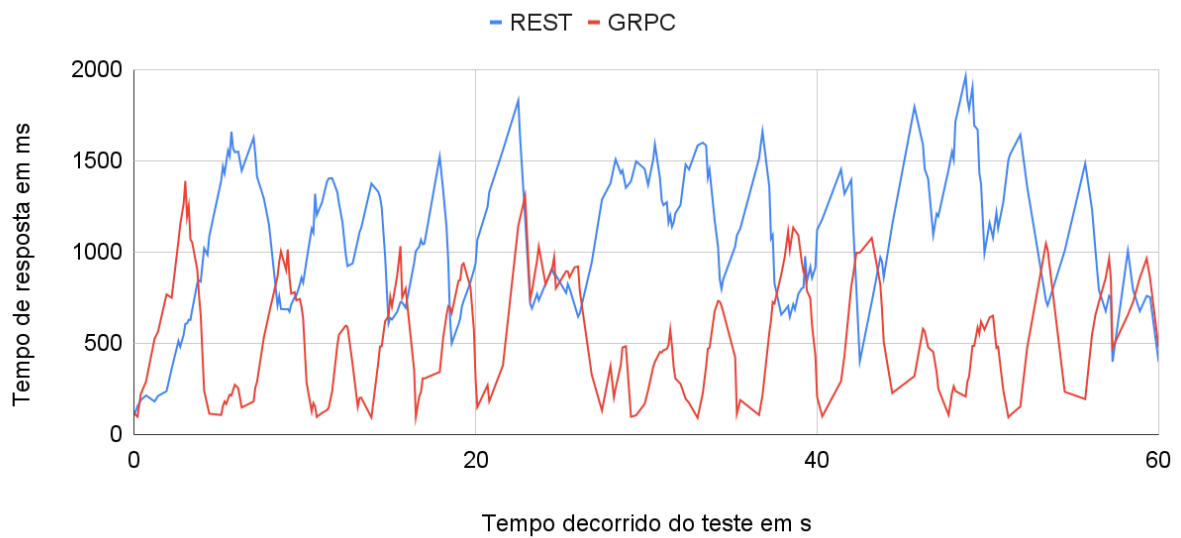


Fonte: O autor.

4.4.1 Amostra de 20 usuários

Para uma amostra de 20 usuários, a Figura 31 ilustra uma comparação do tempo de resposta das duas arquiteturas de comunicação. A arquitetura REST apresentou uma média de tempo de resposta de 1066,5 milissegundos, enquanto que a arquitetura gRPC teve uma média significativamente menor de 527 milissegundos. O desvio padrão foi de 378,81 milissegundos para REST e 320,15 milissegundos para gRPC, indicando que a gRPC foi mais rápido e mais estável em suas respostas.

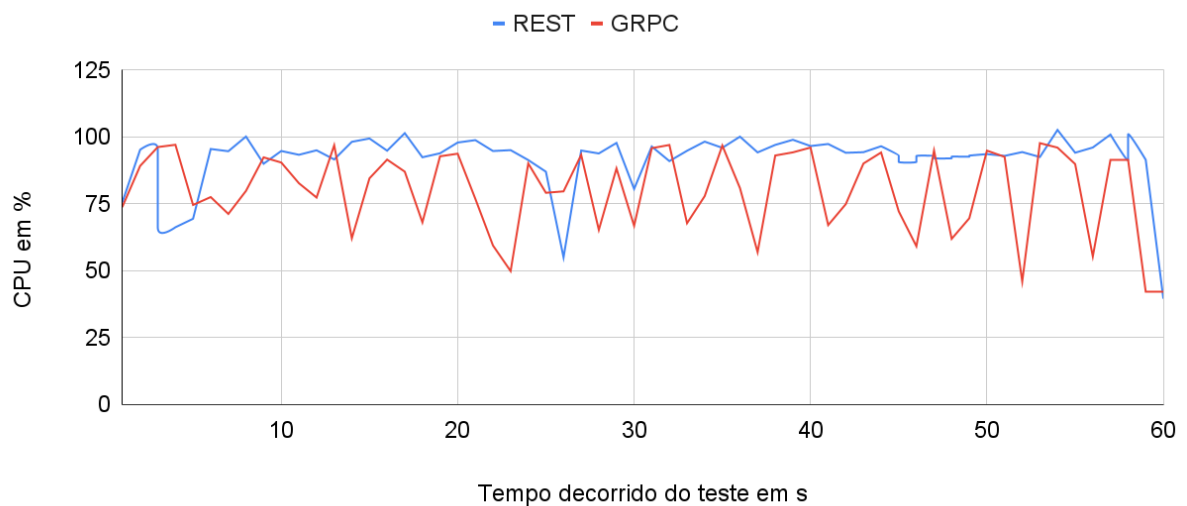
Figura 31 – Tempo de resposta para teste com inserção de um registro em amostra de 20 usuários.



Fonte: O autor.

Observa-se também que o consumo de CPU na arquitetura gRPC é menor, com uma média de 83,5%, em comparação com os 94,39% da arquitetura REST. Isso demonstra que a arquitetura gRPC é mais eficiente em termos de processamento.

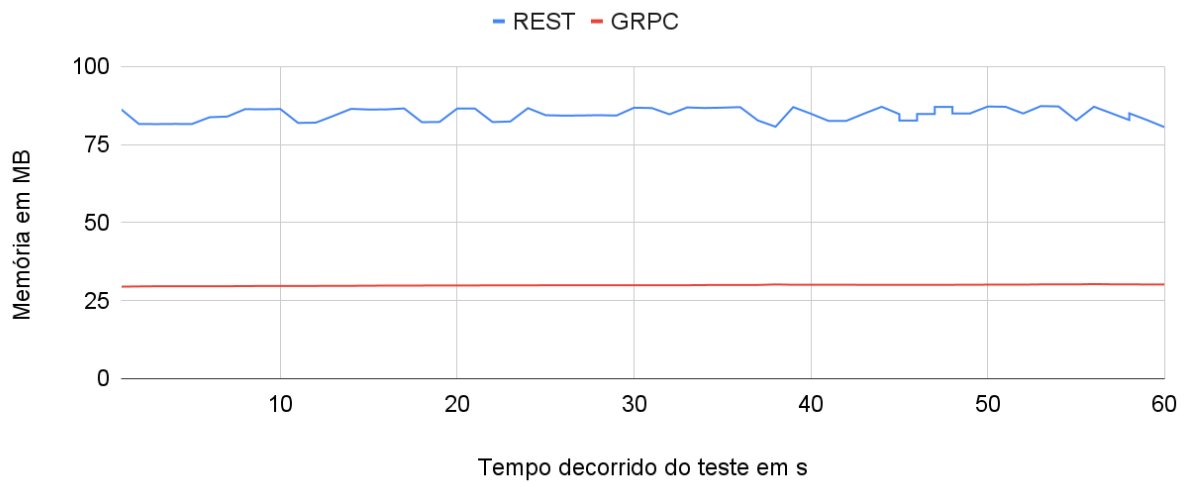
Figura 32 – Consumo de CPU para teste com inserção de um registro em amostra de 20 usuários.



Fonte: O autor.

Em termos de consumo de memória, a arquitetura REST apresentou uma média de 84,89MB, enquanto que a arquitetura gRPC teve uma média de 29,93MB. Novamente, a arquitetura gRPC mostra uma vantagem clara em termos de eficiência.

Figura 33 – Consumo de memória para teste com inserção de um registro em amostra de 20 usuários.

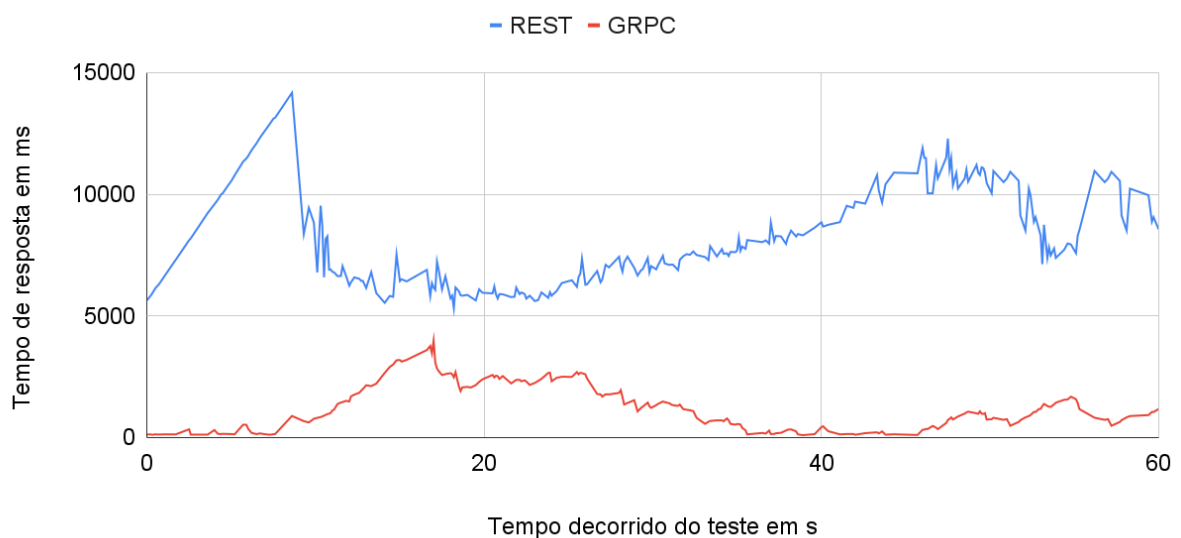


Fonte: O autor.

4.4.2 Amostra de 100 usuários

Para uma amostra de 100 usuários, a Figura 34 ilustra uma comparação do tempo de resposta das duas arquiteturas de comunicação. A arquitetura REST apresentou uma média de tempo de resposta de 7578 milissegundos, enquanto que a arquitetura gRPC teve uma média significativamente menor de 1012 milissegundos. O desvio padrão foi de 1948,78 milissegundos para REST e 916,42 milissegundos para gRPC, indicando que a gRPC é mais rápida e mantém uma maior consistência nas respostas mesmo com um maior número de usuários.

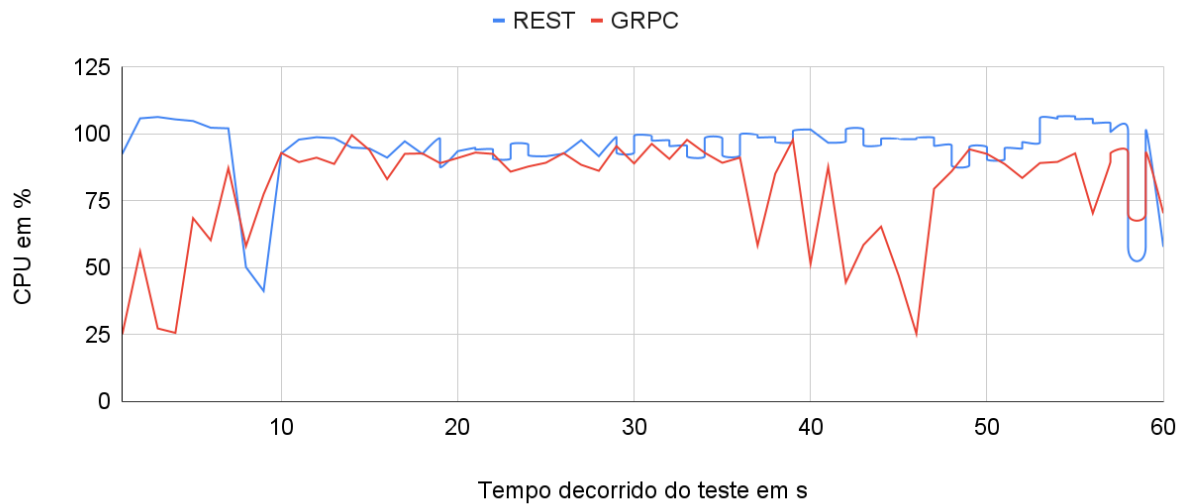
Figura 34 – Tempo de resposta para teste com inserção de um registro em amostra de 100 usuários.



Fonte: O autor.

O consumo de CPU na arquitetura gRPC continuou a ser significativamente menor, com uma média de 88,66%, em comparação com os 96,78% da arquitetura REST. Isso reforça a eficiência superior da gRPC.

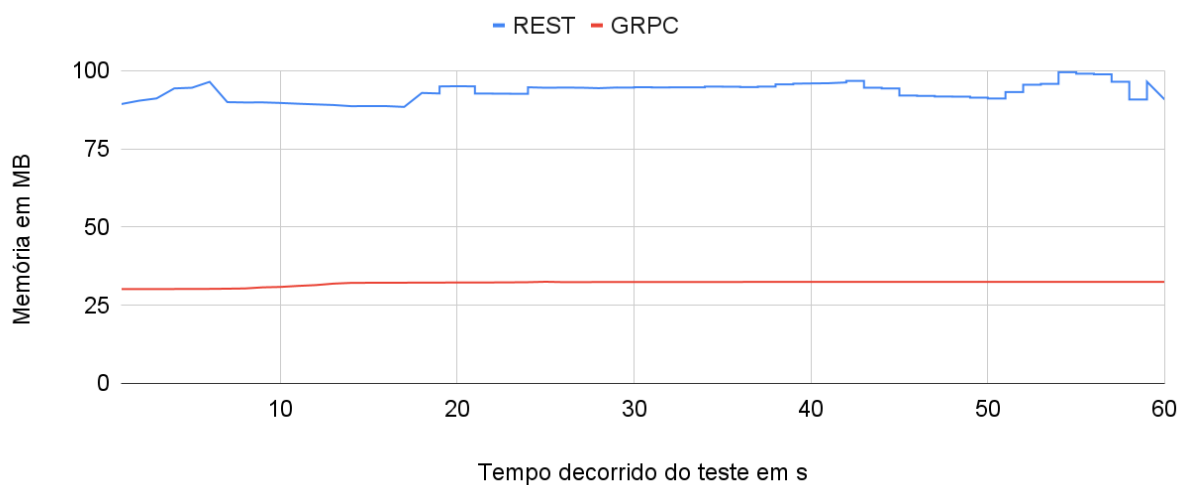
Figura 35 – Consumo de CPU para teste com inserção de um registro em amostra de 100 usuários.



Fonte: O autor.

Finalmente, para a métrica de consumo de memória, a arquitetura REST apresentou uma média de 94,4MB, enquanto que a arquitetura gRPC teve uma média de 32,44MB, mostrando novamente uma vantagem clara para a gRPC em termos de eficiência de recursos.

Figura 36 – Consumo de memória para teste com inserção de um registro em amostra de 100 usuários.



Fonte: O autor.

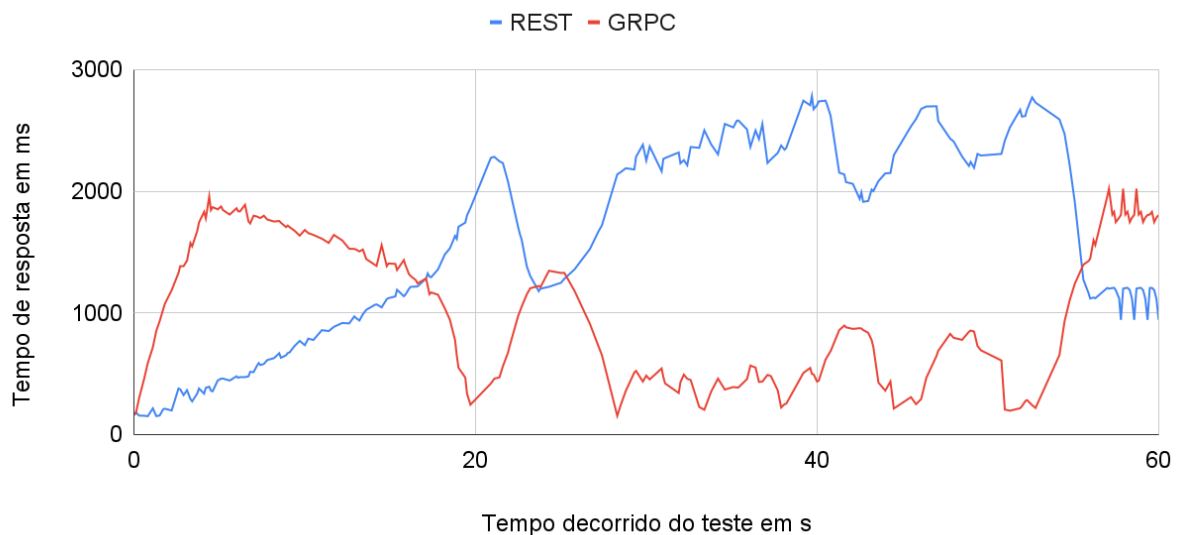
4.5 Teste 4: Inserção de Dez Registros

Para este teste, cada requisição inseriu uma lista de registros contendo o mesmo *payload*, garantindo assim a igualdade dos dados.

4.5.1 Amostra de 20 usuários

Para uma amostra de 20 usuários, a Figura 37 ilustra uma comparação do tempo de resposta das duas arquiteturas de comunicação. A arquitetura REST apresentou uma média de tempo de resposta de 1297.5 milissegundos, enquanto que a arquitetura gRPC teve uma média significativamente menor de 1037 milissegundos. O desvio padrão foi de 803.84 milissegundos para REST e 581.86 milissegundos para gRPC, indicando que a gRPC é mais rápida e também mais estável em suas respostas.

Figura 37 – Tempo de resposta para teste com inserção de uma lista de livros em amostra de 20 usuários.

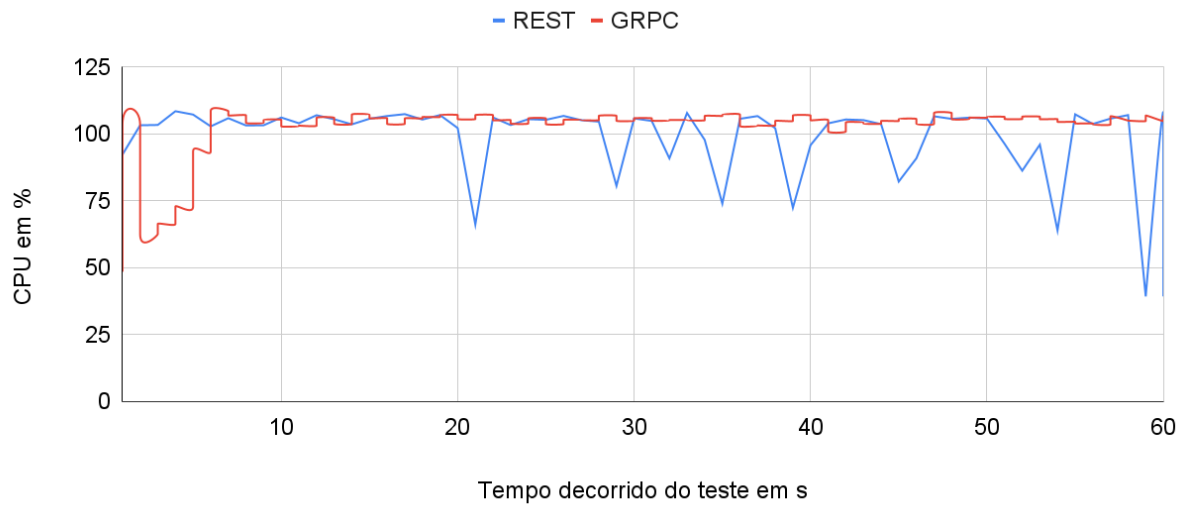


Fonte: O autor.

Observa-se também que o consumo de CPU na arquitetura gRPC é ligeiramente maior, com uma média de 105,05%, em comparação com os 104,74% da arquitetura REST. A diferença, embora pequena, sugere que ambas as arquiteturas têm um uso intenso de CPU para esta carga.

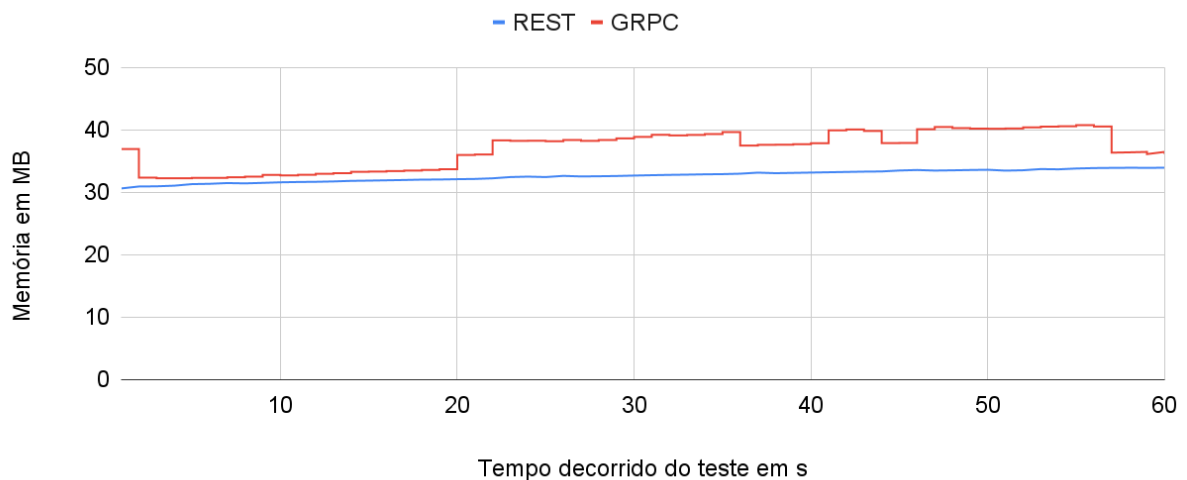
Em termos de consumo de memória, a arquitetura REST apresentou uma média de 32,72MB, enquanto que a arquitetura gRPC teve uma média de 37,89MB, mostrando um uso ligeiramente maior de memória pela gRPC.

Figura 38 – Consumo de CPU para teste com inserção de uma lista de livros em amostra de 20 usuários.



Fonte: O autor.

Figura 39 – Consumo de memória para teste com inserção de uma lista de livros em amostra de 20 usuários.

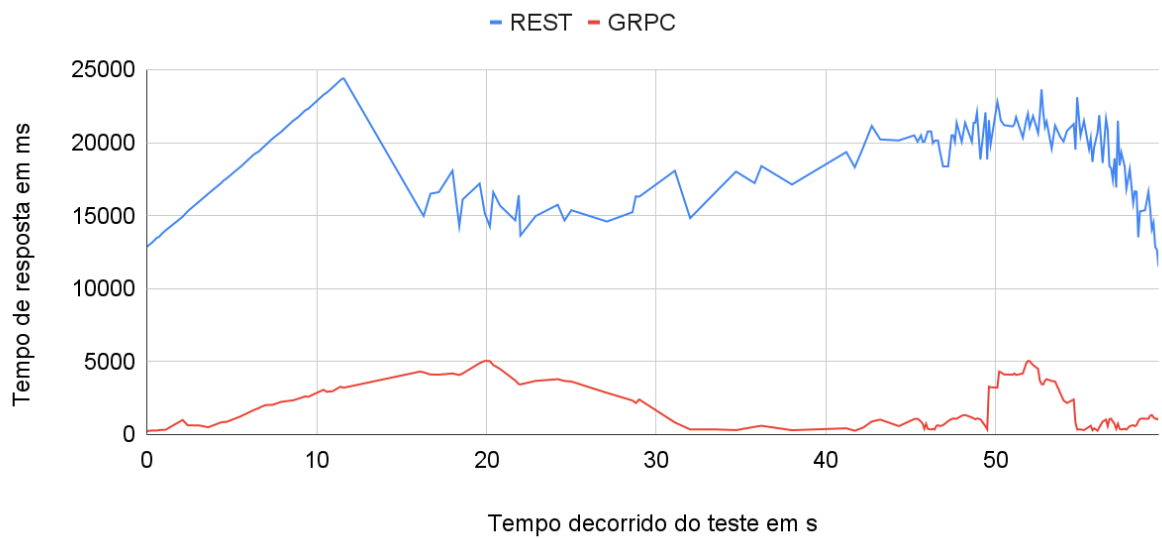


Fonte: O autor.

4.5.2 Amostra de 100 usuários

Para uma amostra de 100 usuários, a Figura 40 ilustra uma comparação do tempo de resposta das duas arquiteturas de comunicação. A arquitetura REST apresentou uma média de tempo de resposta de 19501 milissegundos, enquanto que a arquitetura gRPC teve uma média significativamente menor de 1080 milissegundos. O desvio padrão foi de 2924,18 milissegundos para REST e 1511,53 milissegundos para gRPC, indicando novamente que a gRPC é mais rápida e mais estável em suas respostas mesmo com um maior número de usuários.

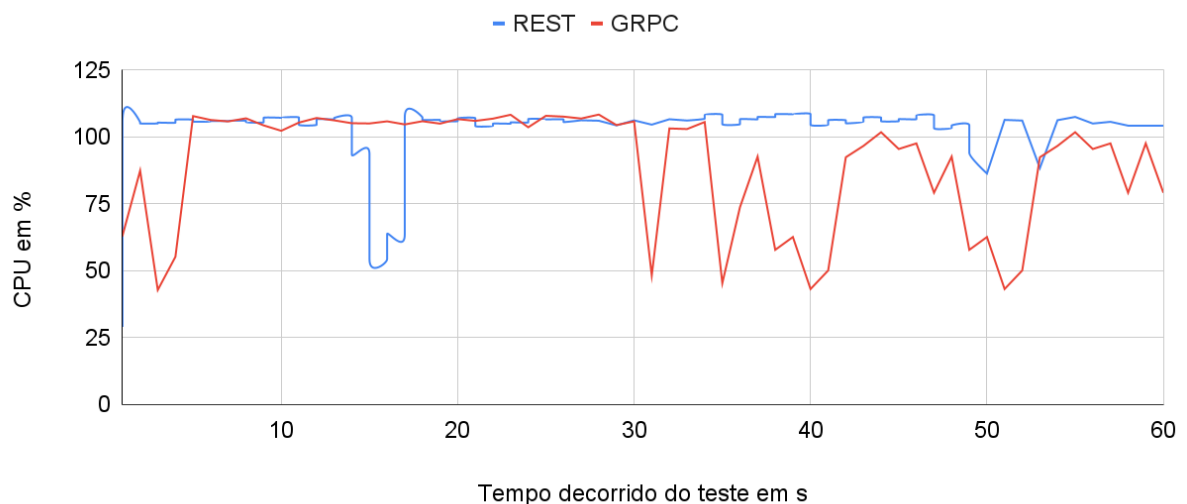
Figura 40 – Tempo de resposta para teste com inserção de uma lista de livros em amostra de 100 usuários.



Fonte: O autor.

O consumo de CPU na arquitetura gRPC continuou a ser menor, com uma média de 101,65%, em comparação com os 105,82% da arquitetura REST, reforçando a eficiência superior da gRPC.

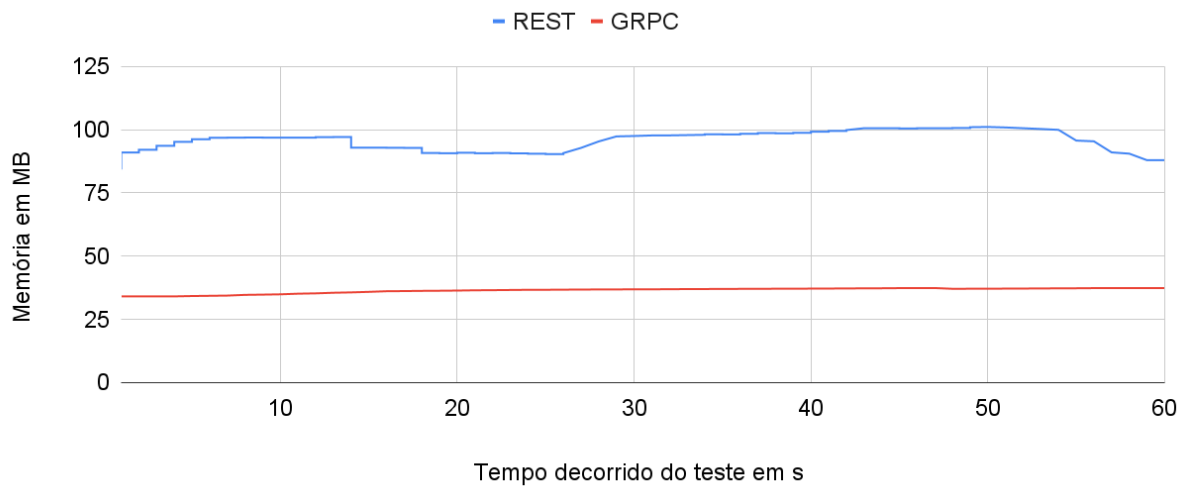
Figura 41 – Consumo de CPU para teste com inserção de uma lista de livros em amostra de 100 usuários.



Fonte: O autor.

Finalmente, para a métrica de consumo de memória, a arquitetura REST apresentou uma média de 96,88MB, enquanto que a arquitetura gRPC teve uma média de 36,89MB, mostrando novamente uma vantagem clara para a gRPC em termos de eficiência de recursos.

Figura 42 – Consumo de memória para teste com inserção de uma lista de livros em amostra de 100 usuários.



Fonte: O autor.

4.6 Considerações Finais

Os resultados dos testes obtidos nesse seção entre as arquiteturas de comunicação REST e gRPC permitiram identificar as qualidades e defeitos de cada arquitetura de maneira mais clara.

Figura 43 – Resultados dos testes de tempo de resposta para REST e gRPC

	Nº de Usuários	Média REST	Desvio Padrão REST	Média gRPC	Desvio Padrão gRPC
Teste 1	20	31.84	5.78	1.34	1.55
Teste 1	100	167.20	51.75	1.38	2.68
Teste 2	20	347.75	100.29	105.29	74.36
Teste 2	100	1911.6	524.66	24.4	162.48
Teste 3	20	1066.5	378.81	527	320.15
Teste 3	100	7578	1948.78	1012	916.42
Teste 4	20	1297.5	803.84	1037	581.86
Teste 4	100	19501	2924.18	1080	1511.53

Fonte: O autor.

Nos testes de obtenção de um registro, a arquitetura gRPC apresentou um desempenho superior em todos os cenários. Com uma média de tempo de resposta significativamente menor (até 96% menor), menor uso de CPU (aproximadamente 53% menor) e menor consumo de memória (cerca de 16% menor), a gRPC demonstrou ser mais eficiente e estável comparada à REST. Esse padrão de desempenho foi mantido tanto para amostras de 20 usuários quanto para amostras de 100 usuários.

Nos testes de obtenção de cinco mil registros, a gRPC novamente mostrou superioridade. A média de tempo de resposta da gRPC foi substancialmente menor (aproximadamente 70% menor) do que a da REST, e a consistência nas respostas também foi maior.

Além disso, o consumo de CPU e memória foi consistentemente menor na arquitetura gRPC, reforçando sua eficiência mesmo em cenários de maior carga.

Figura 44 – Resultados dos testes de uso de CPU para REST e gRPC

	Nº de Usuários	Média REST	Desvio Padrão REST	Média gRPC	Desvio Padrão gRPC
Teste 1	20	60.9	7.72	28.11	3.69
Teste 1	100	61.11	10.51	28.99	3.35
Teste 2	20	74.76	6.87	65.89	5.01
Teste 2	100	81.93	7.70	27.43	16.02
Teste 3	20	94.39	10.90	83.5	15.42
Teste 3	100	96.78	12.40	88.66	19.72
Teste 4	20	104.74	14.13	105.05	9.91
Teste 4	100	105.82	11.47	101.65	21.53

Fonte: O autor.

Os testes de inserção, tanto de um registro quanto de cinco mil registros, confirmaram a superioridade da gRPC. Em ambos os casos, a gRPC apresentou tempos de resposta menores e uso mais eficiente dos recursos de CPU e memória. Mesmo com um número maior de usuários, a arquitetura gRPC manteve um desempenho superior em comparação à REST.

Figura 45 – Resultados dos testes de uso de memória para REST e gRPC

	Nº de Usuários	Média REST	Desvio Padrão REST	Média gRPC	Desvio Padrão gRPC
Teste 1	20	30.34	1.16	25.56	0.08
Teste 1	100	73.08	1.43	28.32	0.27
Teste 2	20	88.46	1.54	63.33	3.74
Teste 2	100	88.84	1.40	60.68	2.77
Teste 3	20	84.89	2.04	29.93	0.20
Teste 3	100	94.4	2.78	32.44	0.79
Teste 4	20	32.72	0.90	37.89	2.95
Teste 4	100	96.88	3.96	36.89	1.07

Fonte: O autor.

Com base nos resultados obtidos, conseguimos analisar que a arquitetura gRPC é mais adequada para cenários que demandam alta performance e eficiência no uso de recursos. A REST, embora mais simples e amplamente utilizada, apresentou tempos de resposta maiores e maior consumo de recursos, o que pode ser um fator limitante em aplicações de alta demanda.

Portanto, para aplicações que exigem baixos tempos de resposta e alta eficiência no uso de CPU e memória, o uso da arquitetura gRPC pode vir a ser uma melhor escolha. No entanto, na hora da escolha também devemos considerar outros fatores como a facilidade de implementação, compatibilidade com sistemas existentes e requisitos específicos do projeto.

5 Conclusão

Este trabalho apresentou diferentes cenários para uma comparação de desempenho entre as arquiteturas REST e gRPC. O objetivo principal consistiu em trazer as vantagens propostas por cada arquitetura e verificar em quais cenários uma apresenta vantagens em relação à outra. O primeiro cenário analisou a obtenção de um registros, permitindo verificar a eficiência de resposta e consumo de recursos em consultas simples. O segundo cenário explorou a obtenção de muitos registros, testando a capacidade das arquiteturas em lidar com grandes volumes de dados. O terceiro cenário abordou a inserção de registros com *payload* com apenas um registro, enquanto o quarto cenário considerou a inserção com um *payload* contendo 10 registros, avaliando a eficiência de escrita de dados em diferentes tamanhos de carga.

Cada cenário realizou testes com dois volumes de dados diferentes para evitar a ocorrência de viés nos testes. O ambiente de testes foi configurado para garantir a consistência e a repetibilidade dos resultados, utilizando contêineres Docker para que o resultado não fosse distorcido pelo *hardware* do computador.

Concluiu-se que, em ambientes onde é necessário alto desempenho e eficiência no uso de recursos, a arquitetura gRPC oferece uma grande vantagem de desempenho, principalmente em termos de tempo de resposta, uso de CPU e consumo de memória. Por outro lado, a arquitetura REST, embora mais simples e amplamente adotada, apresentou maiores tempos de resposta e maior consumo de recursos, o que pode ser um fator limitante em aplicações de alta demanda e na escalabilidade das mesmas.

5.1 Trabalhos Futuros

Embora este trabalho tenha fornecido uma comparação detalhada entre as arquiteturas REST e gRPC, ainda existem muitos detalhes a serem explorados em pesquisas futuras. Para aprofundar este trabalho, é possível coletar mais métricas de desempenho e realizar uma comparação mais abrangente. Além disso, elaborar diferentes cenários de banco de dados para entender como as APIs se comportam com esquemas e tecnologias diferentes pode oferecer um diferente ponto de vista.

Uma limitação desse trabalho foi a utilização de apenas uma linguagem de programação para implementar ambas as arquiteturas. Uma melhoria futura seria implementar as APIs utilizando diferentes linguagens de programação para eliminar o viés tecnológico. Adicionalmente, explorar a integração com outros sistemas e avaliar a interoperabilidade das arquiteturas REST e gRPC em contextos mais complexos pode contribuir ainda mais

com a pesquisa.

Por fim, esse trabalho contribui para a literatura relacionada ao desempenho de arquiteturas de APIs, oferecendo dados empíricos que podem ajudar a escolha entre REST e gRPC em diferentes contextos de aplicação. A escolha da arquitetura ideal deve ser baseada em uma análise cuidadosa das necessidades do projeto, considerando tanto as métricas de desempenho quanto outros fatores relevantes para o sucesso da implementação.

Referências

BECKHAUSER, L. *Performance Comparasion Between Rest and gRPC*. [S.l.]: GitHub, 2024. <<https://github.com/beckhauser/performance-comparison-between-rest-and-grpc>>.

BERG, J.; REDI, D. M. *Benchmarking the request throughput of conventional API calls and gRPC: A Comparative Study of REST and gRPC*. [S.l.]: KTH Royal Institute of Technology, 2023.

BITTENCOURT, A. L. R. *Comparação de desempenho entre gRPC e REST: um estudo de caso*. 2021. Trabalho de Conclusão de Curso, Universidade de Brasília. Disponível em: <https://bdm.unb.br/bitstream/10483/31169/1/2021_AndreLuizRamosBittencourt_tcc.pdf>.

BOLANOWSKI, M. et al. Efficiency of rest and grpc realizing communication tasks in microservice-based ecosystems. *arXiv preprint arXiv:2208.00682*, 2022.

BRITANNICA, E. *HTTP / Definition, Meaning, Versions, & Facts / Britannica*. 2024. Accessed: 2024-07-22. Disponível em: <<https://www.britannica.com/technology/HTTP>>.

BUFFERS, P. *Overview*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://protobuf.dev/>>.

BUFFERS, P. *Overview*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://protobuf.dev/overview/>>.

BUZHIN, I. et al. Comparative analysis of the rest and grpc used in the monitoring system of communication network virtualized infrastructure. *T-Comm-Telekommunikatsii i Transport*, Obshchestvo s ogranichennoy otvetstvennost'yu Izdatel'skiy dom Media pabliher, v. 17, n. 4, p. 50–55, 2023.

DOCKER. *Docker overview*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://docs.docker.com/guides/docker-overview/>>.

DOCS mdn web. *API*. 2024. Acessado em 9 de maio de 2024. Disponível em: <<https://developer.mozilla.org/en-US/docs/Glossary/API>>.

DOCS mdn web. *HTTP request methods*. 2024. Acessado em 9 de maio de 2024. Disponível em: <<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>>.

DOCS mdn web. *Quais os tipos de API e para que servem?* 2024. Acessado em 9 de maio de 2024. Disponível em: <<https://developer.mozilla.org/en-US/docs/Web/HTTP>>.

FIELDING, R. T. *Architectural styles and the design of network-based software architectures*. [S.l.]: University of California, Irvine, 2000.

GROUND, S. *What is HTTP/2? - Beginner Guide*. 2024. Accessed: 2024-08-14. Disponível em: <<https://www.siteground.com/kb/what-is-http2/#:~:text=HTTP%2F2%20also%20introduces%20a,performance%2C%20security%2C%20and%20efficiency%20of>>.

- GRPC. *Core concepts, architecture and lifecycle*. 2022. Acessado em 8 de novembro de 2023. Disponível em: <<https://grpc.io/docs/what-is-grpc/core-concepts/>>.
- GRPC. *Introduction to gRPC*. 2023. Acessado em 8 de novembro de 2023. Disponível em: <<https://grpc.io/docs/what-is-grpc/introduction/>>.
- HAMILTON, T. *What is JMeter? Introduction and Uses*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://www.guru99.com/introduction-to-jmeter.html>>.
- JMETER, A. *Apache JMeter™*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://jmeter.apache.org/>>.
- JSON. *Introducing JSON*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://www.json.org/json-en.html>>.
- OSS, Z. *jmeter-grpc-request*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://github.com/zalopay-oss/jmeter-grpc-request>>.
- PLUGINS, J. *Response Times Over Time*. 2024. Acessado em 23 de junho de 2024. Disponível em: <<https://jmeter-plugins.org/wiki/ResponseTimesOverTime/>>.
- ŚLIWA, M.; PAŃCZYK, B. Performance comparison of programming interfaces on the example of rest api, graphql and grpc. *Journal of Computer Sciences Institute*, v. 21, p. 356–361, 2021.